Assignment 1

Underactuated Robotics and Manipulation

Acrobots, Cart-Poles, Quadrotors, Walking Models,
Legged Robots, and an Introduction to Manipulation

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

Abstract

This report documents the four parts of Assignment 1: three chapters of *Underactuated Robotics* (acrobots, cart-poles and quadrotors; simple models of walking and running; highly-articulated legged robots) and the introduction of *Robotic Manipulation*. I ran the supplied notebooks, reproduced their main results, and added parameter and robustness studies where they were useful. The visible autograders pass: 13/13 and 12/12 for the cart-pole notebooks, 13/13 for the hopper, 16/16 for the two footstep planners, and 70/70, 50/50 and 50/50 for the manipulation exercises. The report gives the steps behind these numbers, the plots that show the behaviour, and the code that produced them. The less obvious cases—saturation, non-minimum-phase zeros, the narrow compass-gait basin, and the partial stability of the SLIP apex map—are discussed with the numerical evidence rather than only stated as conclusions.

Contents

1 Introduction **6**
- 1.1 Individual work and scope . . . 6
- 1.2 Environment and code . . . 8
- 1.3 Summary of results . . . 8

I Underactuated Systems **9**

2 Underactuated Ch. 3: Acrobots, Cart-Poles, and Quadrotors **10**
- 2.1 Example 3.1: Dynamics of the Acrobot . . . 10
- 2.2 Example 3.2: Dynamics of the Cart-Pole System . . . 11
- 2.3 Example 3.5: LQR for the Acrobot and Cart-Pole . . . 12
- 2.4 Example 3.6: LQR for Quadrotors . . . 13
- 2.5 Section 3.8: Exercises 3.1–3.4 . . . 13
  - 2.5.1 Exercise 3.1: linearisation and balancing . . . 13
  - 2.5.2 Exercise 3.2: cart-pole URDFs . . . 15
  - 2.5.3 Exercise 3.3: controllability . . . 16
  - 2.5.4 Exercise 3.4: stability and linearisation . . . 18
- 2.6 Implementation notes . . . 18
- 2.7 Discussion . . . 20
- 2.8 Answers to the notebook questions . . . 20

3 Underactuated Ch. 4: Simple Models of Walking and Running **22**
- 3.1 Example 4.3: Numerical simulation of the rimless wheel . . . 22
- 3.2 Example 4.4: Numerical simulation of the compass gait . . . 23
- 3.3 Examples 4.5 and 4.6: SLIP simulation and the apex-to-apex map . . . 23
- 3.4 Section 4.6: The One-Dimensional Hopper . . . 25
- 3.5 Implementation notes . . . 26
- 3.6 Discussion . . . 27
- 3.7 Answers to the notebook questions . . . 27

4 Underactuated Ch. 5: Highly-articulated Legged Robots **29**
- 4.1 Example 5.1: The ZMP Planner . . . 29
- 4.2 Example 5.2: LittleDog Gait Optimization . . . 29
- 4.3 Section 5.8: Exercises 5.1–5.3 . . . 29
  - 4.3.1 Exercise 5.1: Footstep Planning via MIQP . . . 31
  - 4.3.2 Exercise 5.2: Understanding the Zero-Moment Point . . . 31
  - 4.3.3 Exercise 5.3: Footstep Planning via GCS . . . 31

4.4 Implementation notes . . . 32
4.5 Discussion . . . 32
4.6 Answers to the notebook questions . . . 33

II Manipulation **34**

5 Manipulation Ch. 1: Introduction to Manipulation and Drake **35**
5.1 Examples 1.1 and 1.2: Teleoperation in 2D and 3D . . . 35
5.1.1 Background: Example 1.1 – Teleoperation in 2D . . . 35
5.1.2 Example 1.2: Teleoperation in 3D . . . 35
5.2 Example 1.3: System Diagrams . . . 36
5.3 Section 1.7: Exercises 1.1–1.4 . . . 36
5.4 Implementation notes . . . 38
5.5 Discussion . . . 39
5.6 Answers to the notebook questions . . . 39

6 Conclusions **41**

A Code and Reproduction **43**
A.1 Selected code . . . 43

List of Figures

2.1 Acrobot with the ±10 N m saturation. The green run starts with a 0.05 rad shoulder error; the red run also has a 0.03 rad elbow error and saturates. The elbow moves away from zero before the shoulder returns, which is the non-minimum-phase undershoot. 11

2.2 Cart-pole success rate versus force limit at $\delta = 0.1$. At 20 N the pole still balances in 90% of trials; at 10 N the rate is 64%, and at 5 N only 36%. Without a limit the pole is easy; the actuator is the binding constraint. 12

2.3 Normalised controllability singular values and transmission zeros. The acrobot's smallest normalized singular value is about 1.4×10^{-4}; the cart-pole's is about 2.1×10^{-2}. Both have right-half-plane zeros: the acrobot at +2.05 and +2.28, the cart-pole at +4.43. 13

2.4 Planar quadrotor sensitivity. With 0.25 N wind the steady-state offset is 0.073 m; with 2 N it is 0.554 m. The other panels show the expected Q/R trade-off. 14

2.5 Closed-loop angle histories for the six initial states of Exercise 3.1. Cases 3 ($\theta = \pi/2$) and 4 ($\theta = 0$) leave the upright region; the linear controller was designed for the tangent at π. 15

2.6 Frame convention used in Exercise 3.2. The joint origin is expressed in the parent frame (red axes), while the child link's inertial and visual origins are expressed in the child frame (blue axes). The second link is drawn rotated for clarity; at $\theta_1 = 0$ it coincides with the first link. 16

2.7 Double-pendulum cart-pole under LQR. The angle errors at 5 s are 0.014 and 0.0007 rad, but the cart is still moving; the slow pair of poles has a time constant of about 1.3 s. 17

2.8 Edges of the discrete double integrator for $u = -1, 0, +1$. The source nodes are solid; destinations outside the 3×3 grid are dashed. The two-step construction $s_{0,0} \to (0, i) \to (i, j)$ is the path used in the text. 18

2.9 Phase portraits around the upright equilibrium for $b = 0$ (top) and $b = 1$ (bottom). The stable manifold separates initial conditions that fall left from those that fall right; damping does not remove the saddle. 19

3.1 Rimless-wheel return map for three slopes. Only slopes above about 0.067 rad have a fixed point above the step threshold; at 0.05 rad the wheel cannot complete a step. 23

3.2 Compass gait: phase portrait and step-to-step map. The step points converge to the fixed point after the initial transient; perturbations outside the narrow basin fall. . . 24

3.3 SLIP apex-to-apex map. The stable fixed point is at 0.8776 m with $P' = 0.42$; the unstable one is at 0.9163 m with $P' = 1.29$. Filled points are stable fixed points. . . 24

3.4 Hopper energy and preload for $h = 1$ m. The two energy drops are the touchdown and lift-off impacts; the preload pulses at the bottom restore the energy. 26

4.1 ZMP planner: CoM and ZMP trajectories for the nominal footstep plan. The default weights give a ZMP tracking error of 0.153 m; after a velocity push the feedback run returns to 1.5×10^{-5} m. 30
4.2 LittleDog gait: optimized contact schedule and body trajectory. The stride is 0.6875 s and the CoM height varies between 0.131 and 0.148 m. 30
4.3 Section 5.8 footstep plans on the benchmark terrains. The MIQP costs are 4.159 and 5.672 for terrains A and B; the GCS plans use 10 edges. 32

5.1 Teleoperation data flow. The sliders command a full pose, diff-IK converts it to joint positions, and the measured pose is fed back. 35
5.2 A system diagram in the spirit of Example 1.3. The station is a nested diagram; expanding the IIWA controller shows the position controller and model driver inside. 36
5.3 Inverted-pendulum simulations for the two verification conditions. The final angles are 0.0052 and -0.1074 rad, matching the autograder. 37
5.4 Joint positions of the IIWA14 under the proportional controller. The residuals are below 0.06 rad; the P controller does not compensate gravity, so a small steady-state offset remains. 38

List of Tables

1.1 Assignment 1 scope. 6
1.2 Coverage of the assignment items: every required Example and Section, the report section that discusses it, the corresponding code, and the headline result. Each row appears again as a numbered section heading in the body. 7
1.3 Automatic grading results. 8

Chapter 1

Introduction

1.1 Individual work and scope

All code, simulations, figures and analysis in this report were done by me. The course supplied the notebooks and the textbook chapters; where a notebook already contained a working example, I ran it, changed parameters, and recorded what happened. Where an exercise required new code, I wrote it and checked the result with the supplied autograder. The report is one document rather than four separate ones so that the same ideas—linearisation, controllability, return maps, and contact planning—can be compared across chapters.

Assignment 1 covers four textbook chapters. The first three belong to *Underactuated Robotics*; the last one is the introduction of *Robotic Manipulation*. The required examples and exercise sections are summarised in Table 1.1.

Table 1.1: Assignment 1 scope.

Textbook chapter	Examples	Exercise section
Ch. 3 Acrobots, Cart-Poles, and Quadrotors	3.1, 3.2, 3.5, 3.6	Section 3.8, Exercises 3.1–3.4
Ch. 4 Simple Models of Walking and Running	4.3, 4.4, 4.5, 4.6	Section 4.6, one-dimensional hopper
Ch. 5 Highly-articulated Legged Robots	5.1, 5.2	Section 5.8, three exercises
Ch. 1 Introduction (Manipulation)	1.2, 1.3	Section 1.7, Exercises 1.1–1.4

The file names in Table 1.2 are relative to the chapter folders `Ch-3/`, `Ch-4/`, `Ch-5/` and `Ch-1/`; the complete tree is in Appendix A.

The examples are the supplied demonstrations: the code is already written and the task is to run it, explain what is observed, and, where possible, study the effect of changing a parameter. The exercise sections are solved from scratch; the report includes the derivations, the important code, and the relevant plots. Every Example starts from the model and the linearisation, and every Exercise section states the steps before quoting the answer, because the numerical result alone

Table 1.2: Coverage of the assignment items: every required Example and Section, the report section that discusses it, the corresponding code, and the headline result. Each row appears again as a numbered section heading in the body.

Assignment item	§	Code / notebook	Headline result
Example 3.1 Dynamics of the Acrobot	§2.1	`exp1_analysis/exp1_analysis.py`; `Exp3'1-acrobot.ipynb`	98% without, 63% with ± 10 N m
Example 3.2 Dynamics of the Cart-Pole System	§2.2	`exp3_2_analysis/cp_analysis.py`; `Exp3'2-cartpole.ipynb`	100% unlimited, 36% at 5 N
Example 3.5 LQR for Acrobot and Cart-Pole	§2.3	`exp3_5_analysis/lqr_comparison.py`; `Exp3'5-acrobot.ipynb`, `Exp3'5-cartpole.ipynb`	condition numbers 7.2×10^3 vs 9.8×10^1
Example 3.6 LQR for Quadrotors	§2.4	`exp3_6_analysis/quadrotor_analysis.py`; `Exp3'6-quadrotor.ipynb`	3D hover in 1.08 s
Section 3.8 Exercises 3.1–3.4	§2.5	`Ch.3 Exercises/cartpole_balancing.ipynb`, `cartpoles_urdf.ipynb`; `exercises_analysis/ex3_*.py`	13/13 and 12/12
Example 4.3 Numerical simulation of the rimless wheel	§3.1	`exp4_analysis/rimless_wheel_analysis.py`; `Exp4'3-rimless-wheel.ipynb`	multiplier $\cos^2(2\alpha) = \frac{1}{2}$
Example 4.4 Numerical simulation of the compass gait	§3.2	`exp4_analysis/compass_gait_analysis.py`; `Exp4'4-compass-gait.ipynb`	14 steps in 10 s, period 0.735 s
Example 4.5 Simulation of the SLIP model	§3.3	`exp4_analysis/slip_analysis.py`; `Exp4'5-6-slip.ipynb`	stable $z^\star = 0.8776$ m, $P' = 0.420$
Example 4.6 Numerical SLIP apex-to-apex map	§3.3	`exp4_analysis/slip_analysis.py`; `Exp4'5-6-slip.ipynb`	unstable $z_u = 0.9163$ m, $P' = 1.288$
Section 4.6 Exercise: One-Dimensional Hopper	§3.4	`exp4_analysis/one_d_hopper_analysis.py`; `Exp4'6-one_d_hopper.ipynb`	13/13; height within 6 mm
Example 5.1 The ZMP Planner	§4.1	`exp5_analysis/example51_zmp.py`; `Exp5'1-zmp-planner.ipynb`	tracking error 0.153 m $\rightarrow 1.5 \times 10^{-5}$ m
Example 5.2 LittleDog Gait Optimization	§4.2	`exp5_analysis/littledog_analysis.py`; `Exp5'2-littledog.ipynb`	stride 0.6875 s
Section 5.8 Exercises 5.1–5.3	§4.3	`exp5_analysis/footstep_analysis.py`, `zmp_exercise_analysis.py`; `Exp5'8-footstep-planning*.ipynb`	MIQP 16/16, GCS 16/16
Example 1.2 Teleoperation in 3D	§5.1.2	`Exp1.1-1.2-teleoperation.ipynb` (in `Ch-1/examples/`)	3D teleoperation runs headless
Example 1.3 System Diagrams	§5.2	textbook interactive diagram (no code required)	diagram structure reproduced in Figure 5.2
Section 1.7 Exercises 1.1–1.4	§5.3	`Exp1.7.2` (Drake systems), `Exp1.7.3` (physics), `Exp1.7.4` (HardwareStation)	70/70, 50/50, 50/50

does not show how it was obtained. The derivations follow the course lectures [6] and the textbook sections listed at the start of each chapter.

1.2 Environment and code

All computations use Drake 1.58.0, the `underactuated` package 2026.9.8, and the `manipulation` package 2026.10.17 in the Python environment `/Users/leo/venvs/drake/bin/python`. The notebooks were executed headlessly with `COLAB_TESTING=1`; this prevents the MeshCat visualizer from blocking on a browser connection. The pinned `drake_models` package is used for the IIWA14, the WSG gripper and the YCB object in the manipulation chapter.

The complete code accompanies the report as `Assignment_1_Code.zip`. The files are listed in Appendix A. For the reader who wants to reproduce a specific result, the appendix gives the exact commands.

1.3 Summary of results

Table 1.3 collects the automatic grading results. They are useful as a compact check that the code matches the expected behaviour; the discussion in the following chapters explains what the numbers mean.

Table 1.3: Automatic grading results.

Chapter	Item	Result
Ch. 3	`cartpole_balancing.ipynb`	13/13
Ch. 3	`cartpoles_urdf.ipynb`	12/12
Ch. 4	`one_d_hopper.ipynb`	13/13
Ch. 5	MIQP footstep planning	16/16
Ch. 5	GCS footstep planning	16/16
Ch. 1	Drake Systems Fundamentals	70/70
Ch. 1	Physics Simulation and Robot Loading	50/50
Ch. 1	HardwareStation and Advanced Scenarios	50/50

Part I

Underactuated Systems

Chapter 2

Underactuated Ch. 3: Acrobots, Cart-Poles, and Quadrotors

This chapter covers Examples 3.1, 3.2, 3.5 and 3.6 and Section 3.8 of *Underactuated Robotics* [1]. The corresponding reading is §3.1–3.2 (models and dynamics), §3.4–3.5 (LQR, controllability and the PBH test) and §3.8 (exercises), together with the course lecture [6]. The examples use the notebooks in `Ch-3/Ch.3-examples/`; the exercises are in `Ch-3/Ch.3 Exercises/`.

2.1 Example 3.1: Dynamics of the Acrobot

The acrobot is a two-link planar robot whose shoulder is passive and whose elbow is driven by a torque $u = \tau$ [1]. With $q = [\theta_1, \theta_2]^\top$ and $\mathbf{x} = [\theta_1, \theta_2, \dot\theta_1, \dot\theta_2]^\top$, the manipulator equations are

$$M(q)\ddot q + C(q,\dot q)\dot q + G(q) = \begin{bmatrix} 0 \\ u \end{bmatrix}.$$

The upright equilibrium is $\mathbf{x}^\star = [\pi, 0, 0, 0]^\top$. At this point the linearisation obtained in the notebook is

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 12.629 & -12.693 & -0.172 & 0.3015 \\ -14.749 & 29.612 & 0.3015 & -0.6033 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ -3.0147 \\ 6.0332 \end{bmatrix},$$

with open-loop eigenvalues $\{-6.492, -2.249, +2.231, +5.734\}$. Two positive eigenvalues confirm that the upright position is unstable. The controllability matrix $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \mathbf{A}^3\mathbf{B}]$ has rank four, so the linearised system is controllable even though the robot is underactuated. The singular values of $\mathcal{C}$ are $[511.7, 123.1, 0.0807, 0.0715]$: the largest is about 7×10^3 times the smallest, so some directions are only weakly controllable even though the rank is four. I first read "rank four" as "easy to control"; the Monte-Carlo runs below show the opposite. In the simulation the weak directions appear as a large initial elbow motion. The shoulder moves appreciably only after the elbow has built up momentum, which is what the small singular value predicts.

With $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 1$, the LQR gain is

$$D = [\,278.44,\ 112.29,\ 119.72,\ 56.83\,], \qquad u = D(\mathbf{x} - \mathbf{x}^\star),$$

and the closed-loop poles are -10.126, -4.238 and $-2.242 \pm 0.039j$. A 100-trial study with initial state $\mathbf{x}_0 = \mathbf{x}^\star + 0.05\,\mathcal{N}(0, I_4)$ gives a 98% success rate without saturation and 63% with the ± 10 N m

actuator limit. The successful trajectories show a pronounced non-minimum-phase undershoot: the elbow first moves away from the target before the whole robot returns.

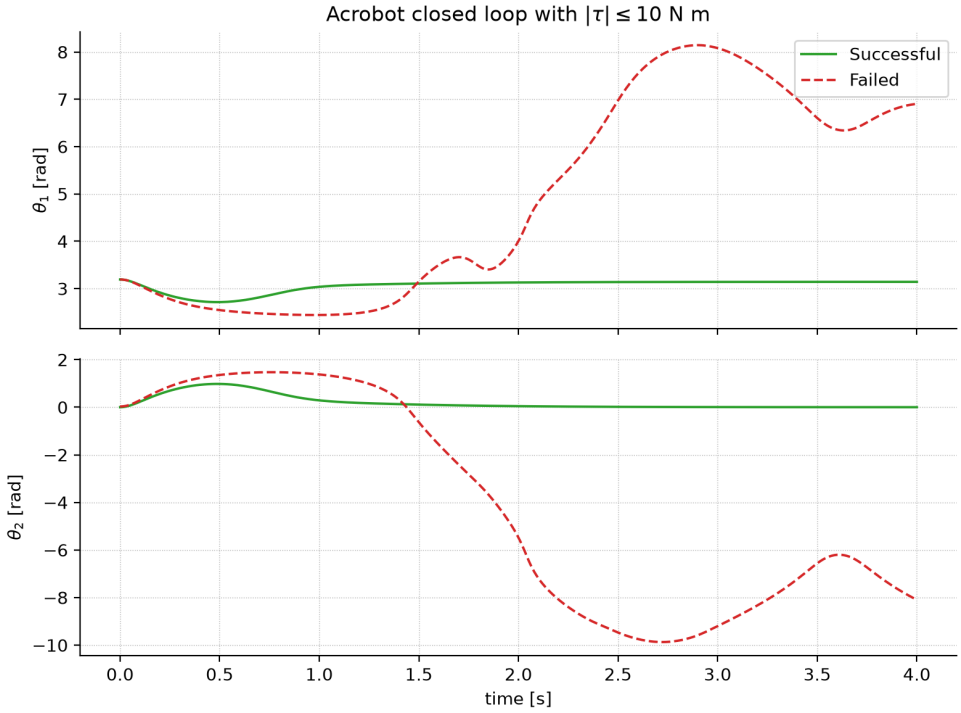


Figure 2.1: Acrobot with the ± 10 N m saturation. The green run starts with a 0.05 rad shoulder error; the red run also has a 0.03 rad elbow error and saturates. The elbow moves away from zero before the shoulder returns, which is the non-minimum-phase undershoot.

Two parameters are especially important. Increasing the distal mass m_2 from 0.5 to 4 kg reduces the minimum controllability singular value by a factor of about 35 and the success rate from 98% to 10%; the peak torque grows from 5 N m to more than 300 N m. Changing R from 1 to 100 reduces the peak torque only from 11.2 to 9.7 N m, because the unstable open-loop poles impose a finite gain even in the expensive-control limit. In practice, the acrobot needs a swing-up phase to enter the LQR region, and the actuator limit must be included in the assessment.

2.2 Example 3.2: Dynamics of the Cart-Pole System

The cart-pole has state $\mathbf{x} = [x, \theta, \dot{x}, \dot{\theta}]^\top$ and input $u = F_x$. For the model used in the notebook, $m_c = 10$ kg, $m_p = 1$ kg and $l_c = 0.5$ m, the linearisation at $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$ is

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0.981 & 0 & 0 \\ 0 & 21.582 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0.1 \\ 0.2 \end{bmatrix}.$$

The open-loop eigenvalues are $0, 0, \pm 4.6456$. The two zero eigenvalues come from the cart position, which behaves as an integrator; the pair ± 4.6456 is the unstable pole. The controllability matrix is full rank, with singular values $[4.326, 4.326, 0.0907, 0.0907]$. The transfer function from u to x has a right-half-plane zero at $+4.43$, so the cart must first move in the opposite direction before it can

bring the pole back. This is the same non-minimum-phase mechanism as in the acrobot, seen more clearly in the cart position.

With $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 1$,

$$D = [\,3.1623,\ -254.3925,\ 9.7692,\ -55.1774\,],$$

and the closed-loop poles are -4.7206, -4.5739 and $-0.3820 \pm 0.3761j$. The two fast poles bring the pole back; the slow pair is the cart returning to the origin. The original notebook does not limit the force, and a 100-trial study with $\mathbf{x}_0 = \mathbf{x}^\star + 0.1\,\mathcal{N}(0, I_4)$ balances the pole in 100% of the trials. The median peak force is about 17 N, with a 90th percentile near 40 N. When the force is limited, the picture changes: 20 N gives 90% pole-balancing success, 5 N gives 36%, and 2 N fails in most trials because the non-minimum-phase recovery needs a larger control authority.

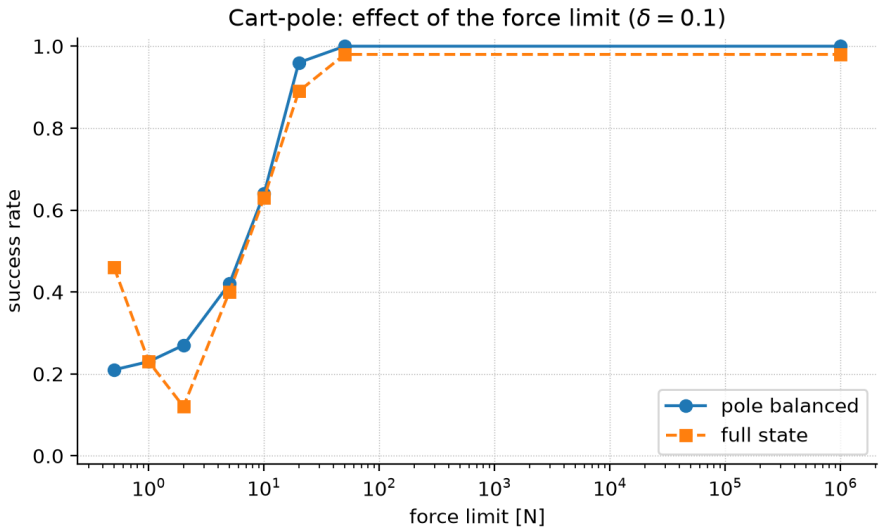


Figure 2.2: Cart-pole success rate versus force limit at $\delta = 0.1$. At 20 N the pole still balances in 90% of trials; at 10 N the rate is 64%, and at 5 N only 36%. Without a limit the pole is easy; the actuator is the binding constraint.

The cart mass is the most important physical parameter. From 2 to 20 kg the median peak force grows from about 5 to 32 N, and the full-state success rate falls from 100% to 68%; the pole itself remains balanceable. Increasing R saves force but slows the cart-return mode: for $R = 100$ the slow pole is about -0.12 rad/s, so a 5 s simulation is too short for the cart to return.

2.3 Example 3.5: LQR for the Acrobot and Cart-Pole

The acrobot and the cart-pole are useful together because they have the same control structure but different modal properties. Both are controllable and both are non-minimum phase. The acrobot's condition number is about 150 times larger, which reflects the weaker coupling through the passive shoulder. The cart-pole has two zero eigenvalues and therefore a slow cart mode, while the acrobot has four relatively fast modes. These differences explain why the same LQR weight has a different effect on the two systems: increasing R mainly reduces the torque on the acrobot, but it substantially slows the cart return on the cart-pole.

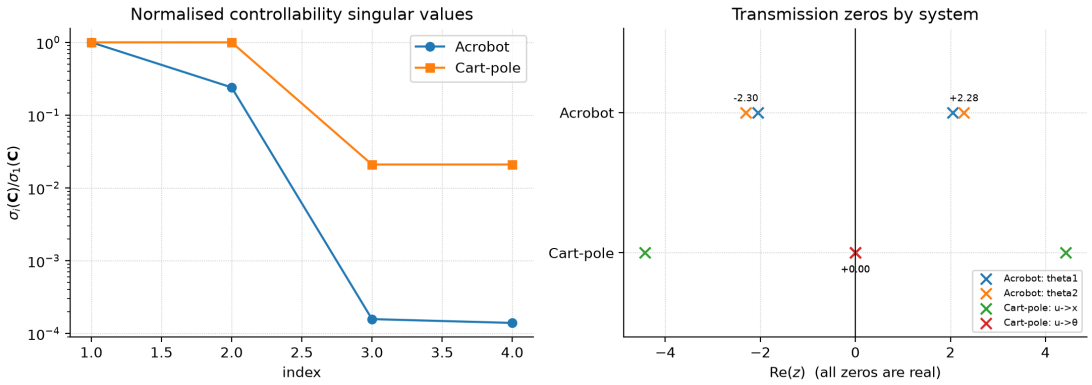


Figure 2.3: Normalised controllability singular values and transmission zeros. The acrobot's smallest normalized singular value is about 1.4×10^{-4}; the cart-pole's is about 2.1×10^{-2}. Both have right-half-plane zeros: the acrobot at $+2.05$ and $+2.28$, the cart-pole at $+4.43$.

2.4 Example 3.6: LQR for Quadrotors

The planar quadrotor has state $\mathbf{x} = [x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}]^\top$ and two thrust inputs. At hover, $u_1 + u_2 = mg$ and $\theta = 0$. The textbook LQR uses

$$\mathbf{Q} = \mathrm{diag}\left(10, 10, 10, 1, 1, \frac{l}{2\pi}\right), \qquad \mathbf{R} = \begin{bmatrix} 0.1 & 0.05 \\ 0.05 & 0.1 \end{bmatrix}.$$

The closed-loop poles are -80.75, -15.86, -6.59, -3.60 and $-2.72 \pm 1.61j$. From an initial tilt of 0.2 rad the controller settles in about 0.06 s with a peak thrust of 5.17 N; ten random initial conditions with standard deviation 0.1 all recover.

The sensitivity study shows the expected trade-offs. Increasing the position weight makes the response faster but raises the peak thrust. Increasing R reduces the peak thrust at the cost of a slower response. A constant wind produces a steady-state offset because LQR has no integral action: the offset is about 0.073 m at 0.25 N and 0.554 m at 2 N. The initial tilt is recovered for all tested values up to 0.7 rad.

The 3D quadrotor has four thrust inputs and six degrees of freedom. It is underactuated, but the hover equilibrium is stabilisable. The LQR is linearised around a point where the total thrust balances gravity, not around zero thrust. From a 0.2 rad roll and 0.1 rad pitch perturbation the official stabilising controller and my LQR both return to hover in about 1.08 s with a peak thrust near 3 N. The sensitivity to Q, R and initial tilt follows the same pattern as in the planar case.

2.5 Section 3.8: Exercises 3.1–3.4

2.5.1 Exercise 3.1: linearisation and balancing

For the cart-pole with $m_c = m_p = l = 1$ and $g = 9.81$, the equations used in the notebook are

$$\ddot{x} = \frac{u + \sin\theta(\dot{\theta}^2 + g\cos\theta)}{1 + \sin^2\theta}, \qquad \ddot{\theta} = \frac{-u\cos\theta - \dot{\theta}^2\cos\theta\sin\theta - 2g\sin\theta}{1 + \sin^2\theta}.$$

The linearisation is done in four steps.

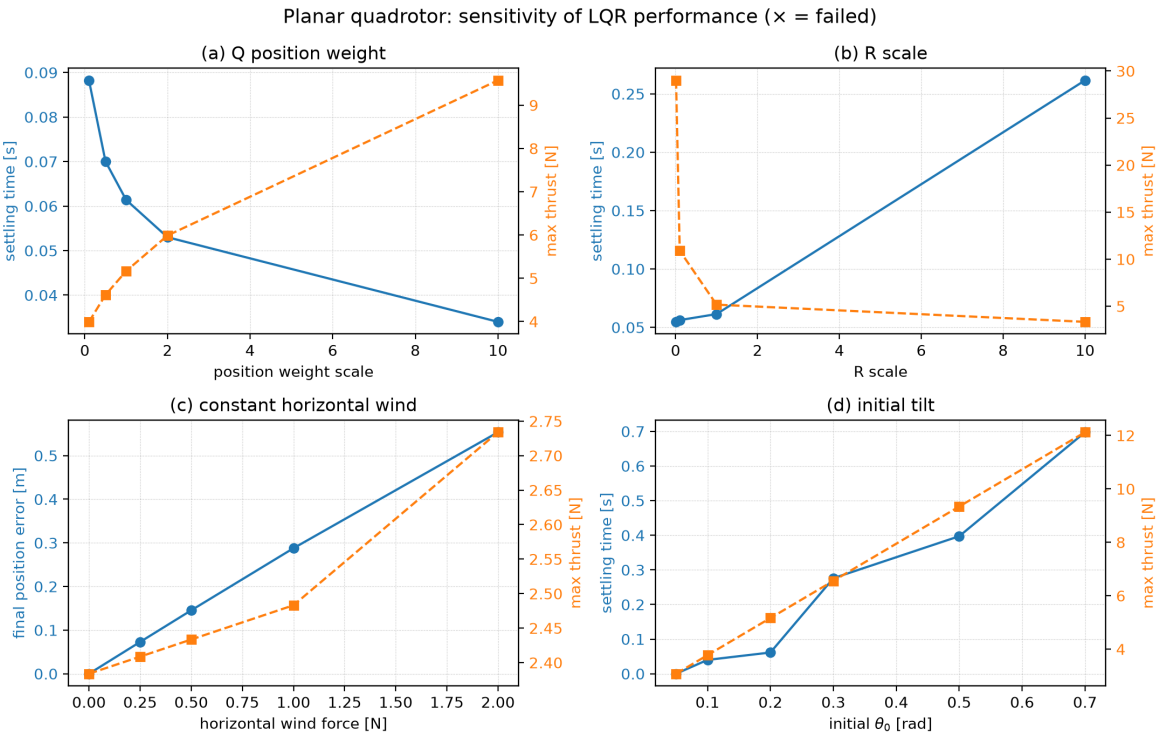


Figure 2.4: Planar quadrotor sensitivity. With 0.25 N wind the steady-state offset is 0.073 m; with 2 N it is 0.554 m. The other panels show the expected Q/R trade-off.

Step 1: shift the angle. The upright equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$. Write $\theta = \pi + \delta$ with small δ, so the state perturbation is $\Delta\mathbf{x} = [0, \delta, 0, \dot{\delta}]^\top$.

Step 2: expand the trigonometric terms. From the addition formulas,

$$\sin(\pi + \delta) = -\sin\delta \approx -\delta, \qquad \cos(\pi + \delta) = -\cos\delta \approx -1,$$

and the denominators satisfy $1 + \sin^2\theta = 1 + \sin^2\delta \approx 1$.

Step 3: drop second-order terms. Substituting into the two nonlinear equations gives, term by term,

$$\ddot{x} = \frac{u + \sin\theta(\dot{\theta}^2 + g\cos\theta)}{1 + \sin^2\theta} \approx \frac{u - \delta\dot{\theta}^2 + g\delta}{1} \approx u + g\delta,$$

because $\delta\dot{\theta}^2$ is second order, and

$$\ddot{\theta} = \frac{-u\cos\theta - \dot{\theta}^2\cos\theta\sin\theta - 2g\sin\theta}{1 + \sin^2\theta} \approx \frac{u - \delta\dot{\theta}^2 + 2g\delta}{1} \approx u + 2g\delta.$$

Step 4: assemble A and B. The state is ordered as $[x, \delta, \dot{x}, \dot{\delta}]$. The two acceleration rows become $[0, g, 0, 0]$ and $[0, 2g, 0, 0]$, and the input enters both accelerations with coefficient one:

$$\mathbf{A}_{\text{lin}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 9.81 & 0 & 0 \\ 0 & 19.62 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B}_{\text{lin}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

The linearisation error is small near the equilibrium and grows quickly with distance. At $\theta = 0.99\pi$ the error is 9×10^{-4}; at $\theta = \pi/2$ it is 26.05; at $\theta = 0$ it is 68.91. The error also depends on the input when $\theta \neq \pi$, because the nonlinear input gains $1/(1+\sin^2\theta)$ and $-\cos\theta/(1+\sin^2\theta)$ are replaced by constants in the linear model. The cart position x does not appear in the dynamics, so it does not affect the error.

With $Q = I_4$ and $R = 1$, the LQR gain is

$$K = [-1.0000,\ 51.0612,\ -2.7221,\ 12.8277], \qquad u = -K(\mathbf{x} - \mathbf{x}^\star),$$

and the closed-loop poles are $-5.0417, -3.9438, -0.5600 \pm 0.4239j$. The six initial states from the notebook are simulated for 5 s; the controller recovers from states 0, 1, 2 and 5:

$$\texttt{system_recovers_from_states} = [0, 1, 2, 5].$$

The first three are close to the upright point. State 5 has $\theta = \pi$ and only a cart offset, which is linearly controllable. States 3 ($\theta = \pi/2$) and 4 ($\theta = 0$) lie outside the region where the linearisation is useful; the controller fails there, as the error table predicts.

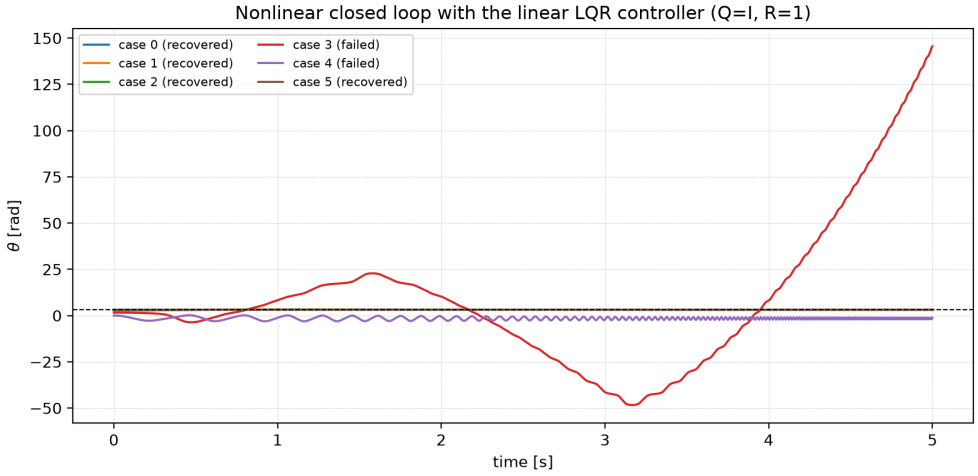


Figure 2.5: Closed-loop angle histories for the six initial states of Exercise 3.1. Cases 3 ($\theta = \pi/2$) and 4 ($\theta = 0$) leave the upright region; the linear controller was designed for the tangent at π.

2.5.2 Exercise 3.2: cart-pole URDFs

The single pendulum URDF has a base of mass 1 at the origin, a pendulum whose ball of mass 1 is at $z = -1$, a spherical visual of radius 0.05 and a cylindrical rod of length 1 and radius 0.01 centred at $z = -0.5$. The joint `theta0` connects the base to the pendulum, with origin $(0, 0, 0)$ and axis $(0, -1, 0)$. The unstable equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$.

The double pendulum needs more care with the frames, and this is the part of Exercise 3.2 where it is easiest to be wrong. In URDF, the `origin` of a `joint` is the pose of the child link frame expressed in the parent link frame; the `origin` of a link's `inertial` and `visual` elements is expressed in that link's own frame (Figure 2.6). So the second joint, whose origin is $(0, 0, -1)$ in `pendulum0`, sits exactly at the ball of the first pendulum, and the second link's ball at $z = +1$ is measured from the second joint, not from the cart. With $\theta_1 = 0$ the second pendulum therefore

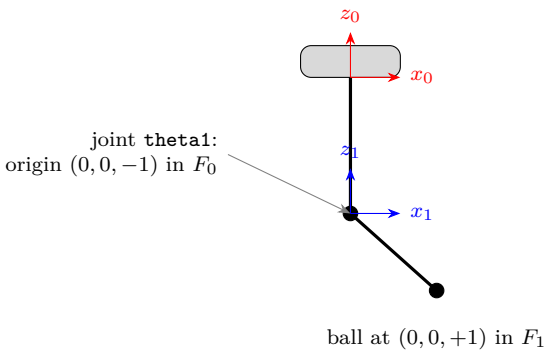


Figure 2.6: Frame convention used in Exercise 3.2. The joint origin is expressed in the parent frame (red axes), while the child link's inertial and visual origins are expressed in the child frame (blue axes). The second link is drawn rotated for clarity; at $\theta_1 = 0$ it coincides with the first link.

points back towards the cart and overlaps the first; with $\theta_0 = \theta_1 = \pi$ both pendula stand upright. The equilibrium is $\mathbf{x}^\star = [0, \pi, \pi, 0, 0, 0]^\top$.

The LQR is designed directly from the nonlinear `MultibodyPlant`. For the single pendulum with $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 1$,

$$K = [-3.1623,\ 61.3987,\ -5.2854,\ 16.1699],$$

with poles $-4.8948, -4.0983, -0.9457 \pm 0.8075j$. After 5 s the angle error is 0.0036 rad and the peak force is 19.3 N. For the double pendulum with $Q = \mathrm{diag}(10, 10, 10, 1, 1, 1)$,

$$K = [3.1623,\ -54.8416,\ -300.4370,\ 7.4384,\ -54.0081,\ -68.9418],$$

with poles $-7.5798, -6.1385, -3.6749, -3.4281, -0.7754 \pm 0.6611j$. The two angle errors after 5 s are 0.0142 and 0.0007 rad, and the peak force is 66.6 N. The autograder gives 12/12.

2.5.3 Exercise 3.3: controllability

The discrete double integrator

$$x_1[n+1] = x_1[n] + x_2[n], \qquad x_2[n+1] = x_2[n] + u[n], \qquad x_1, x_2, u \in \mathbb{Z}$$

maps $s_{i,j}$ to $(i + j, j + u)$. Figure 2.8 shows the edges for $u = -1, 0, +1$. From $s_{0,0}$, one step gives $(0, u_0)$, so $x_1 = 0$ and no state with $i > 0$ is reachable. Two steps give $(u_0, u_0 + u_1)$; choosing $u_0 = i$ and $u_1 = j - i$ reaches any $s_{i,j}$. For an arbitrary start s_{i_1,j_1} and goal s_{i_2,j_2}, the two-step controls are

$$u_0 = i_2 - i_1 - 2j_1, \qquad u_1 = j_2 + j_1 + i_1 - i_2.$$

One step is not enough in general because it fixes $x_1[1] = i_1 + j_1$.

For continuous LTI systems, the four pairs are tested with the controllability matrix. $(\mathbf{A}_1, \mathbf{B}_1)$ and $(\mathbf{A}_2, \mathbf{B}_2)$ have rank one and are not controllable: in the first, x_1 is decoupled from the input; in the second, $z = x_1 - x_2$ satisfies $\dot{z} = z$ independently of u. $(\mathbf{A}_3, \mathbf{B}_3)$ and $(\mathbf{A}_4, \mathbf{B}_4)$ are controllable. For $(\mathbf{A}_4, \mathbf{B}_4)$ the controllability matrix has rank four; the eigenvalues are $0, 0, \pm\sqrt{2}$ and the PBH test is full rank at each eigenvalue. In the second-order interpretation, $(\mathbf{A}_3, \mathbf{B}_3)$ is fully actuated (one degree of freedom, one input), while $(\mathbf{A}_4, \mathbf{B}_4)$ is underactuated ($q = [x, \theta]$, only x actuated). This again separates controllability from full actuation.

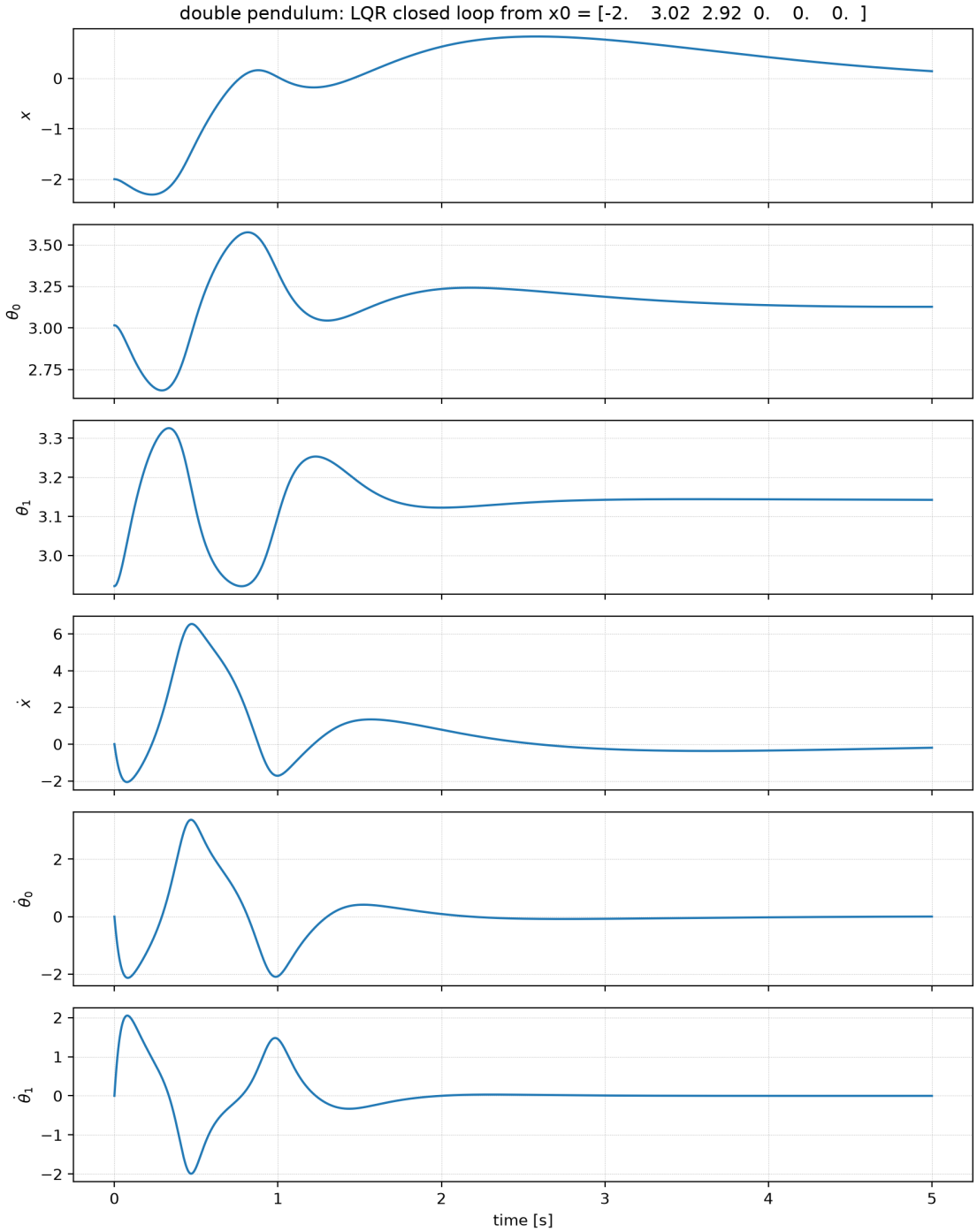


Figure 2.7: Double-pendulum cart-pole under LQR. The angle errors at 5 s are 0.014 and 0.0007 rad, but the cart is still moving; the slow pair of poles has a time constant of about 1.3 s.

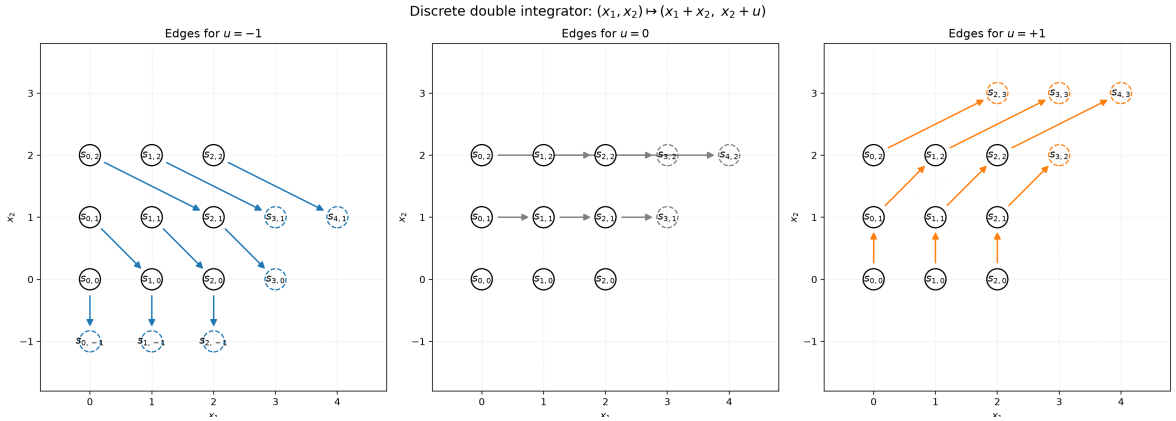


Figure 2.8: Edges of the discrete double integrator for $u = -1, 0, +1$. The source nodes are solid; destinations outside the 3×3 grid are dashed. The two-step construction $s_{0,0} \to (0, i) \to (i, j)$ is the path used in the text.

2.5.4 Exercise 3.4: stability and linearisation

For the pendulum with $m = l = 1$ and damping b,

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -g \sin\theta - b\omega,$$

the equilibrium points are $(0, 0)$ and $(\pi, 0)$, with Jacobian

$$A(\theta^\star) = \begin{bmatrix} 0 & 1 \\ -g\cos\theta^\star & -b \end{bmatrix}.$$

The eigenvalues are $\pm 3.1321j$ for $b = 0$ at the bottom, ± 3.1321 for $b = 0$ at the top, $-0.5 \pm 3.0919j$ for $b = 1$ at the bottom, and $2.6718, -3.6718$ for $b = 1$ at the top.

The stability table follows from these values and from the phase portraits. At the undamped bottom the nonlinear system has closed orbits: it is stable i.s.L. but not asymptotically or exponentially stable. The linearisation is a centre; when the real part is zero the linearisation theorem is inconclusive, and the energy argument is needed. At the top the saddle structure makes both the nonlinear system and its linearisation unstable. With damping, the bottom becomes locally exponentially stable; the top still has one positive eigenvalue, so damping alone cannot stabilise the inverted pendulum.

2.6 Implementation notes

I ran the notebooks with

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=300
```

The `COLAB_TESTING` switch matters here: without it, MeshCat waits for a browser connection and the notebook blocks indefinitely on this machine.

The changes I made to the supplied code are small and listed here so that the numbers can be reproduced:

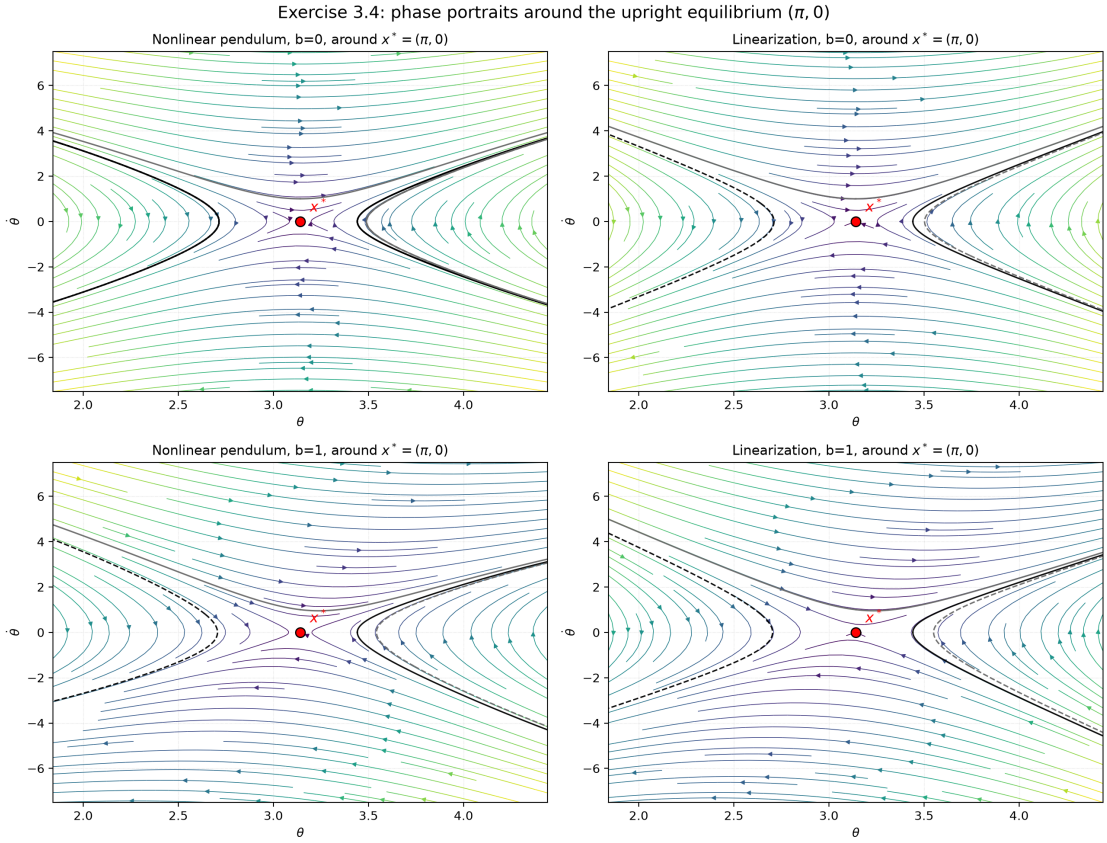


Figure 2.9: Phase portraits around the upright equilibrium for $b = 0$ (top) and $b = 1$ (bottom). The stable manifold separates initial conditions that fall left from those that fall right; damping does not remove the saddle.

- the LQR weights are $Q = \mathrm{diag}(10, 10, 1, 1)$, $R = 1$ unless a study says otherwise; the $R = 100$ case is a separate run;
- the acrobot torque limit is ± 10 N m and the cart-pole force limits are $2, 5, 10, 20$ N;
- the Monte-Carlo studies use 100 trials with a fixed seed, so the percentages in the text are reproducible;
- the quadrotor sensitivity uses initial tilts of 0.05–0.7 rad and constant winds of 0.25 and 2 N.

The acrobot and cart-pole parameters are the defaults from Drake and the `underactuated` package. The supplied controller equations were not changed; the additional code is logging, the loops over random initial conditions, and the sensitivity sweeps in `Ch-3/Ch.3-examples/exp*_analysis/`.

2.7 Discussion

The three examples share a workflow, but they fail in different ways. The acrobot and cart-pole are limited by saturation and by the non-minimum-phase undershoot; the quadrotor is limited by the absence of integral action under a constant wind. The Section 3.8 exercises show that the linear model is local: at $\theta = \pi/2$ the linearisation error is already larger than the field itself.

The models also leave out physics that matters on a real robot. The links are rigid and the joints have no friction or backlash; the actuators are ideal torque or force sources with no rate limit; there is no sensor noise, and the full state is assumed to be measured. The acrobot model has no damping at the passive shoulder, which makes the swing-up problem easier in simulation than on hardware. The linearisation is only useful while δ stays small: the error table in Exercise 3.1 grows from 9×10^{-4} at $\theta = 0.99\pi$ to 26 at $\theta = \pi/2$, so the controller is being asked to use the wrong model long before it visibly fails.

The parameter study points to the distal mass and the actuator limit as the two quantities that decide success. Increasing m_2 from 0.5 to 4 kg reduces the minimum controllability singular value by a factor of 35 and the success rate from 98% to 10%; lowering the cart force limit from 20 to 5 N reduces the success rate from 90% to 36%. A next step would be a swing-up controller for the acrobot and cart-pole (energy-based, as in the textbook), followed by an LQR that respects the saturation, for example by a receding-horizon controller with input constraints. An integral term or a disturbance observer would remove the steady-state offset of the quadrotor under wind. These are outside the scope of this assignment, but they explain why the local LQR results should not be read as global controllers.

2.8 Answers to the notebook questions

What happens when R increases, and why can the gain not go to zero? Increasing R lowers the control effort, but the closed-loop poles must still be moved into the left half-plane. For an unstable open-loop pole at $+5.73$, the gain cannot vanish or the pole stays in the right half-plane. In the acrobot run, $R = 1 \to 100$ reduced the peak torque only from 11.2 to 9.7 N m; the ratio is small because the unstable pole, not the weight, sets the scale.

Why does the linear controller fail at $\theta = \pi/2$? The linearisation is taken at $\theta = \pi$. At $\theta = \pi/2$ the neglected terms are no longer small: the measured linearisation error is 26.05, i.e. about 26 times the magnitude of the vector field at that point. The controller is stabilising the wrong model, and no gain choice fixes that.

Which initial states recover in Exercise 3.1, and why? States 0, 1, 2 and 5. The first three are close to the upright point. State 5 has $\theta = \pi$ with a cart offset, which the linear model can correct because the cart position is an integrator. States 3 ($\theta = \pi/2$) and 4 ($\theta = 0$) are outside the region where the tangent model is meaningful.

What does the PBH test add to the rank test? For $(\mathbf{A}_4, \mathbf{B}_4)$ the controllability matrix already has full rank, so the rank test is enough here. The PBH test $\text{rank}[\lambda_i I - \mathbf{A}_4,\ \mathbf{B}_4] = 4$ at every eigenvalue $0, 0, \pm\sqrt{2}$ checks the same property mode by mode, and it is the test that still works when the controllability matrix is badly conditioned. The pair $(\mathbf{A}_1, \mathbf{B}_1)$ fails both: x_1 is not reached by u at all.

Chapter 3

Underactuated Ch. 4: Simple Models of Walking and Running

This chapter covers Examples 4.3–4.6 and Section 4.6 of *Underactuated Robotics* [2]. The corresponding reading is §4.4–4.6 (rimless wheel, compass gait, SLIP and the hopper), together with the course lecture [6]; the return-map and energy arguments used below follow that text. The notebooks are in `Ch-4/`; the analysis scripts and the numerical sweeps are in `Ch-4/exp4_analysis/`.

3.1 Example 4.3: Numerical simulation of the rimless wheel

The rimless wheel has $n = 8$ spokes, no rim, a point mass at the hip, radius $l = 1$ and slope γ [2]. During stance it behaves as a simple pendulum,

$$\ddot{\theta} = \frac{g}{l} \sin\theta,$$

with energy $E = \frac{1}{2} m l^2 \dot{\theta}^2 + mgl\cos\theta$. At each foot strike the new spoke becomes the stance leg and the collision map is

$$\dot{\theta}^+ = \dot{\theta}^- \cos(2\alpha).$$

For $\alpha = \pi/8$ the kinetic energy is multiplied by $1/2$ at every step. The step-to-step return map is

$$\omega_{n+1} = \cos(2\alpha)\sqrt{\omega_n^2 + 4\frac{g}{l}\sin\alpha\sin\gamma}.$$

A step is possible only above $\omega_1 = \sqrt{2(g/l)[1-\cos(\gamma-\alpha)]}$. The rolling fixed point is

$$\omega^\star = \cot(2\alpha)\sqrt{4\frac{g}{l}\sin\alpha\sin\gamma},$$

and its multiplier is $\cos^2(2\alpha) = 1/2$. The fixed point exists only if it lies above the step threshold.

The slope study gives a clear threshold. At $\gamma = 0.02$ and 0.05 the fixed point is below ω_1 and the wheel cannot sustain a rolling gait; at 0.08, 0.12 and 0.20 the fixed point is valid. The multiplier is $1/2$ in all cases, so the stability of the rolling gait is independent of the slope once the fixed point exists. The fixed point exists only where the gravity input per step can replace the kinetic energy lost at impact; below that slope the map has no valid fixed point.

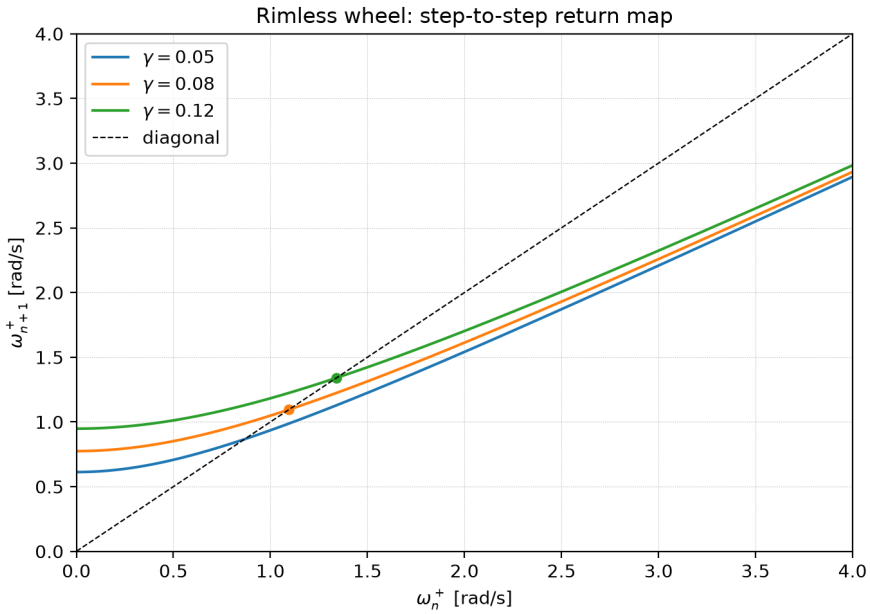


Figure 3.1: Rimless-wheel return map for three slopes. Only slopes above about 0.067 rad have a fixed point above the step threshold; at 0.05 rad the wheel cannot complete a step.

3.2 Example 4.4: Numerical simulation of the compass gait

The compass gait has two point-mass legs of mass $m = 5$ kg and length $l = 1$ m ($a = b = 0.5$ m), and a hip of mass $m_h = 10$ kg. The state is $[\theta_{st}, \theta_{sw}, \dot{\theta}_{st}, \dot{\theta}_{sw}]$. The stance foot is pinned, the swing leg moves freely, and at heel strike the roles of the two legs are swapped. The textbook initial condition is $[0, 0, 0.4, -2.0]^\top$ with zero hip torque and slope $\gamma = 0.0525$ rad.

The phase portrait approaches a closed orbit. A step detector based on the plant's "left support" state counts 14 steps in 10 s, with a mean period of 0.735 s. The post-impact states converge quickly to a neighbourhood of the limit cycle. The basin is narrow: in a four-initial-condition test only the nominal state stayed bounded, and in a 3×3 grid of initial states only 3 of 9 runs reached a sustained gait at the shallow slopes; at $\gamma = 0.08$ and 0.12 none of the nine runs did. The passive gait is therefore a valid demonstration, but not a robust walking controller.

The contrast with the rimless wheel is instructive. The wheel has many spokes and a collision geometry that naturally returns the state to the rolling solution. The compass gait has only two legs, so the swing-leg landing must match the previous step closely. The energy balance can still produce a limit cycle, but the basin is much smaller.

3.3 Examples 4.5 and 4.6: SLIP simulation and the apex-to-apex map

The spring-loaded inverted pendulum models running with a point mass, a massless spring leg and a touchdown angle that is reset during flight. In flight the dynamics are ballistic; in stance,

$$m\ddot{r} - mr\dot{\theta}^2 + mg\cos\theta - k(r_0 - r) = 0, \qquad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} - mgr\sin\theta = 0.$$

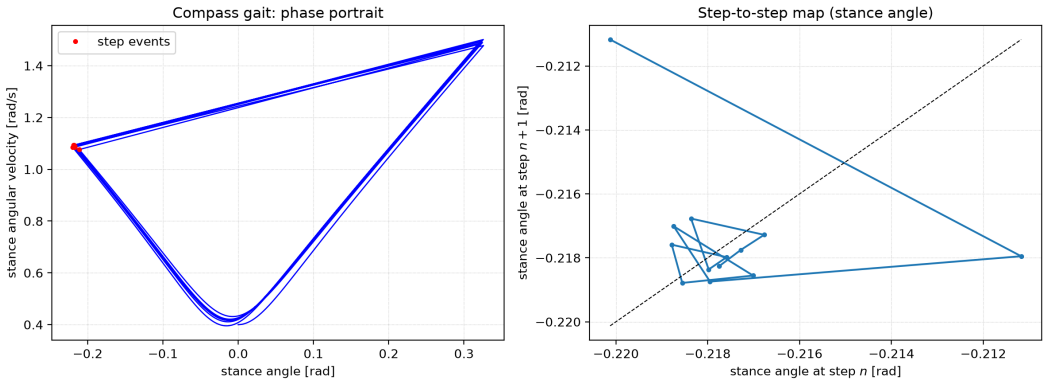


Figure 3.2: Compass gait: phase portrait and step-to-step map. The step points converge to the fixed point after the initial transient; perturbations outside the narrow basin fall.

The mechanical energy

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr\cos\theta + \frac{1}{2}k(r_0 - r)^2$$

is conserved because the leg and foot are massless. Unlike the walking models, there is no impact loss; the spring stores and returns the energy.

At the apex the vertical velocity is zero, and energy conservation makes the horizontal velocity a function of the apex height. The return map is therefore one-dimensional, $z[n+1] = P(z[n])$. For the nominal condition $\theta_{td} = 30°$ and $\tilde{E} = 1.61$ the stable fixed point is $z^\star = 0.8776$ with $P' = 0.420$, and the unstable one is $z_u = 0.9163$ with $P' = 1.288$. The textbook values are 0.8768 and 0.9168, with derivatives 0.488 and 1.346; the difference is due to the finite grid and linear interpolation.

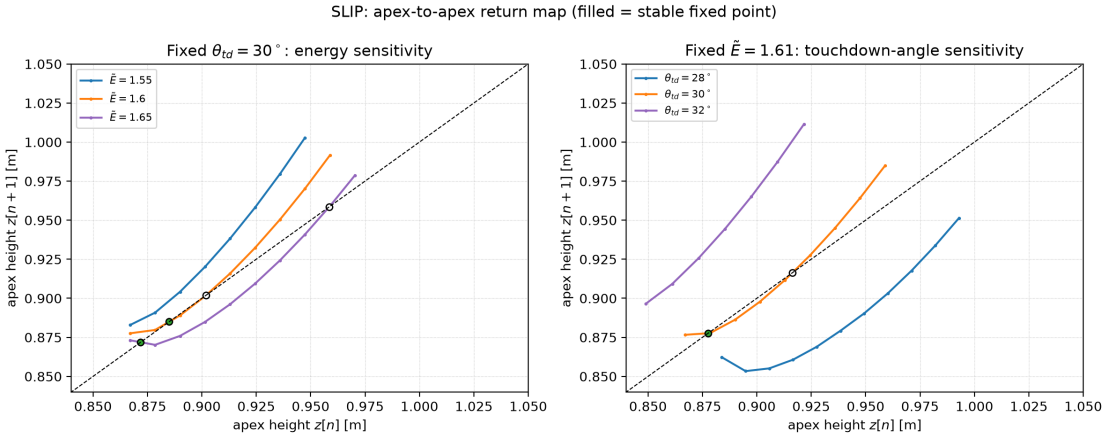


Figure 3.3: SLIP apex-to-apex map. The stable fixed point is at 0.8776 m with $P' = 0.42$; the unstable one is at 0.9163 m with $P' = 1.29$. Filled points are stable fixed points.

The sensitivity study shows a narrow operating window. At 30° a stable fixed point exists for $\tilde{E} = 1.60, 1.61, 1.65$; changing the touchdown angle to 28° or 32° removes the fixed point in the scanned range. The one-dimensional map is obtained on a constant-energy slice, so $|P'| < 1$ proves stability only for perturbations that preserve the energy. Terrain changes and pushes change the

energy and are outside the map; this is the partial stability discussed in the notes. The apex map is useful for one-step controllers, but a real robot still needs energy regulation.

3.4 Section 4.6: The One-Dimensional Hopper

The hopper has a body m_b, a leg m_l, a spring k and a preload $\delta \geq 0$. The state is $[z_b, l, \dot{z}_b, \dot{l}]$. The controller changes the preload twice per cycle: at the bottom it applies $\bar{\delta}$, and at the apex it resets the preload to zero. The mechanical energy is

$$E = \frac{1}{2} m_b \dot{z}_b^2 + \frac{1}{2} m_l (\dot{z}_b + \dot{l})^2 + m_b g z_b + m_l g (z_b + l) + \frac{1}{2} k (l + \delta)^2.$$

The preload follows from five steps. I write them out because the result is easy to quote and hard to reproduce from the final formula alone.

Step 1: energy at the bottom. At the bottom the leg is fully compressed, $l = -z_b$, and $\dot{z}_b = 0$, so both kinetic terms vanish. Before the preload is applied, the potential energy is

$$E(t_{bt}^-) = m_b g z_b + \underbrace{m_l g (z_b + l)}_{=0} + \underbrace{\tfrac{1}{2} k l^2}_{=\frac{1}{2} k z_b^2} = m_b g z_b + \frac{1}{2} k z_b^2.$$

Applying the preload changes the spring rest length from 0 to $\bar{\delta}$; the leg does not move instantaneously, so only the spring term changes:

$$E(t_{bt}^+) = m_b g z_b + \frac{1}{2} k (\bar{\delta} - z_b)^2.$$

Step 2: velocity just before lift-off. The foot stays on the ground until $l = 0$, and the preload is still applied at that instant, so the spring retains $\frac{1}{2} k \bar{\delta}^2$. Subtracting that from the post-preload energy of Step 1 leaves the body's kinetic energy just before the foot leaves the ground:

$$\frac{1}{2} m_b \dot{z}_b^2 (t_{lo}^-) = m_b g z_b + \frac{1}{2} k z_b^2 - k \bar{\delta} z_b.$$

Step 3: inelastic impact at lift-off. The leg has no actuator at the hip and no damper, so when the foot leaves the ground the body and the leg must move together. Momentum is conserved in the collision,

$$m_b \dot{z}_b (t_{lo}^-) = (m_b + m_l) \dot{z}_b (t_{lo}^+) \quad \Longrightarrow \quad \dot{z}_b (t_{lo}^+) = \frac{m_b}{m_b + m_l} \dot{z}_b (t_{lo}^-),$$

and the kinetic energy drops by the factor $m_b / (m_b + m_l)$.

Step 4: energy at the apex. At the apex the body and leg are at rest at height h, and the spring still carries the preload:

$$E(t_{ap}^-) = (m_b + m_l) g h + \frac{1}{2} k \bar{\delta}^2.$$

Step 5: solve for the preload. Equating $E(t_{lo}^{+})$ (Step 2 after the collision of Step 3, plus the spring term $\tfrac{1}{2}k\bar{\delta}^2$) with $E(t_{ap}^{-})$ (Step 4) and solving for $\bar{\delta}$ gives the value used by the controller,

$$\bar{\delta} = \frac{1}{2}z_b + \frac{m_b g}{k} - \frac{(m_b + m_l)^2}{k m_b z_b} g h.$$

The $1/z_b$ term is why the formula is only valid at the bottom: at the apex z_b is large and the required preload would be negative, which is why the controller resets $\delta = 0$ there. If this value is negative the system already has too much energy, and the controller uses $\bar{\delta} = 0$.

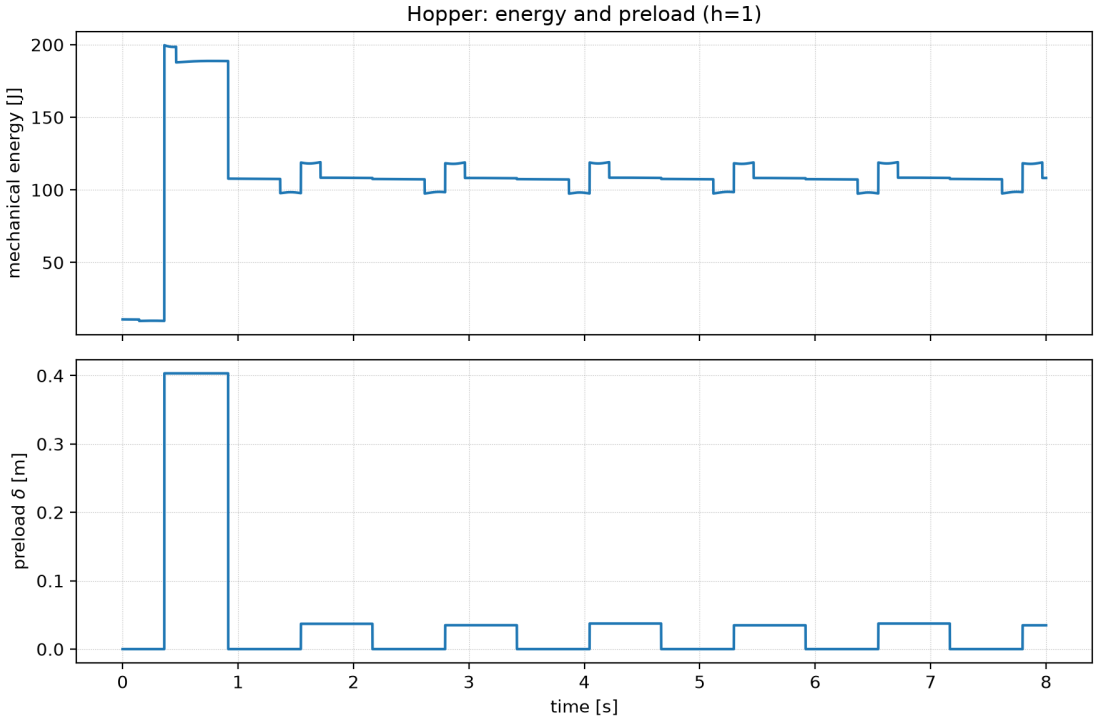


Figure 3.4: Hopper energy and preload for $h = 1$ m. The two energy drops are the touchdown and lift-off impacts; the preload pulses at the bottom restore the energy.

The controller is implemented as a `LeafSystem` that detects the bottom and apex by a sign change of $\dot{z}_b$ and sets the preload accordingly. The autograder gives 13/13. In an 8 s simulation the final-two-second maximum height is within 6 mm of the target for $h = 0.5, 0.75, 1.0, 1.25, 1.5$ m. The maximum preload grows from about 0.024 to 0.302 m over this range, as expected from the energy balance. The controller is also insensitive to the initial body height: starting from $z_b(0) = 0.1$ to 1.0 m, the steady hopping height is the same.

3.5 Implementation notes

I used the supplied rimless-wheel, compass-gait and SLIP notebooks and implemented the hopper controller. Steps in the compass gait are detected from the plant's "left support" state; the rimless return map is computed from the formula in the notebook; the SLIP map uses the notebook's `SpringLoadedInvertedPendulum`. The parameters I varied are:

- rimless wheel slopes $\gamma = 0.02, 0.05, 0.08, 0.12, 0.20$ rad;

- compass-gait slopes $\gamma = 0.0525, 0.08, 0.12$ rad and a 3×3 grid of initial states;
- SLIP energies $\tilde{E} = 1.60, 1.61, 1.65$ and touchdown angles $28°, 30°, 32°$;
- hopper target heights $h = 0.5, 0.75, 1.0, 1.25, 1.5$ m and initial body heights $z_b(0) = 0.1$–1.0 m.

The hopper controller computes the preload at the bottom and resets it at the apex; the autograder gives 13/13. Each hopper run is 8 s and the last 2 s are used for the height statistics. The only implementation difficulty was detecting the event reliably: $\dot{z}_b = 0$ exactly at the apex, so the sign test uses non-strict comparisons on the previous sample and strict comparisons on the current one, which is why `is_bottom_or_apex` has two branches.

3.6 Discussion

The passive walkers and SLIP illustrate two different stability mechanisms. The rimless wheel and the compass gait rely on impact dissipation, and their basins are set by the collision geometry; the compass gait's basin is much narrower because it has only two legs. SLIP has no impact loss, so the apex map is conservative and the stability is only partial. The hopper adds a controller that restores the lost energy.

What the models ignore is substantial. The wheels and legs are point masses on massless links; the contacts are instantaneous and perfectly inelastic, with no foot slip, no rolling resistance and no ground compliance; the terrain is a constant slope. The hopper is one-dimensional, so it cannot fall sideways or trip. These idealisations are what make the return maps one-dimensional and analytically tractable, but they also mean that the predicted basins should be read as upper bounds. The compass gait result is the clearest example: the passive limit cycle exists, yet only 3 of 9 perturbed initial states reached it.

The sensitivity is concentrated in a few parameters. The rimless-wheel gait exists only above a slope threshold (about 0.067 rad for these parameters), and below it the map has no fixed point; the compass gait loses its cycle entirely at $\gamma = 0.08$ and 0.12 in my grid; the SLIP fixed point disappears when the touchdown angle moves from 30° to 28° or 32°. A next step would be energy regulation on top of the apex map (Raibert-style control), a contact model with compliance for the walkers, and a two- or three-dimensional hopper to check whether the one-dimensional controller survives lateral perturbations.

3.7 Answers to the notebook questions

Why does the rimless wheel need a minimum slope? Every foot strike multiplies the kinetic energy by $\cos^2(2\alpha)$, so a fraction $1 - \cos^2(2\alpha)$ is lost at each step. Gravity must replace exactly that amount during stance. The energy added per step is proportional to $\sin\alpha \sin\gamma$, which vanishes as $\gamma \to 0$; below $\gamma \approx 0.067$ rad the added energy is smaller than the impact loss, the fixed point falls below the step threshold ω_1, and the wheel stops.

Why is the compass-gait basin narrower than the rimless wheel's? The wheel has eight identical spokes, so any spoke strike lands the same geometry. The compass gait has two legs, and the swing leg must land at the angle determined by the previous step. A perturbation changes the landing geometry, and the step-to-step map amplifies some of these changes; in my grid only 3 of 9 perturbed states reached the limit cycle.

Why does $|P'| < 1$ not prove stability of SLIP? The apex map is computed on a surface of constant mechanical energy, because the horizontal velocity at the apex is determined by the apex height through energy conservation. A perturbation that changes the energy (a push, a step up) moves the state off that surface, and the one-dimensional derivative says nothing about that direction. This is the partial stability discussed in the notes.

Why reset the preload to zero at the apex? During flight the leg is massless and the spring force cannot change the body's ballistic motion, so the preload has no effect until the next touchdown; keeping it would only waste energy. The formula for $\bar{\delta}$ also contains a $1/z_b$ term, which is meaningless at the apex.

Chapter 4

Underactuated Ch. 5: Highly-articulated Legged Robots

This chapter covers Examples 5.1 and 5.2 and Section 5.8 of *Underactuated Robotics* [3]. The corresponding reading is §5.3 (ZMP and the linear inverted pendulum), the gait-optimisation section for LittleDog, and §5.8 (footstep planning with MIQP and GCS), together with the course lecture [6]. The notebooks are in `Ch-5/Ch.5-examples/`; the analysis scripts are in `Ch-5/exp5_analysis/`.

4.1 Example 5.1: The ZMP Planner

The first example is the ZMP planner for a humanoid [3]. The model is the linear inverted pendulum, with the centre of mass at height z_c and the zero-moment point (ZMP) as the control point. In the notebook the ZMP is tracked by a linear controller; the nominal plan is a sequence of foot placements and a reference ZMP. The CoP identity is checked numerically and holds to machine precision. With the default weights the ZMP tracking error is about 0.153 m; after a velocity push, the feedback policy reduces the final error to 1.5×10^{-5} m. The timing sensitivity study shows that the tracking error grows when the step timing is shortened, because the LIPM has less time to move the CoM between supports.

4.2 Example 5.2: LittleDog Gait Optimization

The second example optimises a walking gait for the LittleDog model. The decision variables are the gait timing and the contact schedule; the constraints come from the centroidal dynamics and the foot geometry. The notebook uses SNOPT. In the executed version the optimisation converges and gives a stride period of about 0.6875 s and a CoM height between 0.131 and 0.148 m. The result is a periodic gait with alternating support phases. The gait is therefore an optimisation problem over contacts; the feedback controller only tracks the resulting trajectory.

4.3 Section 5.8: Exercises 5.1–5.3

The section formulates footstep planning in two ways. In the MIQP, a binary variable assigns each foot to one stepping stone, the continuous foot positions are constrained to the corresponding stone, and the objective penalises the sum of the step lengths. In the GCS formulation the terrain is decomposed into convex regions, and the problem becomes a shortest path through a graph of

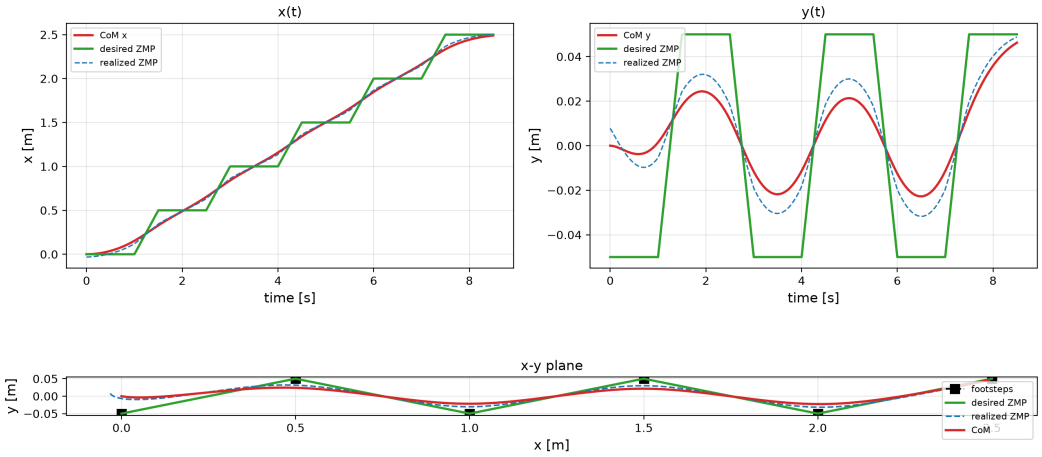


Figure 4.1: ZMP planner: CoM and ZMP trajectories for the nominal footstep plan. The default weights give a ZMP tracking error of 0.153 m; after a velocity push the feedback run returns to 1.5×10^{-5} m.

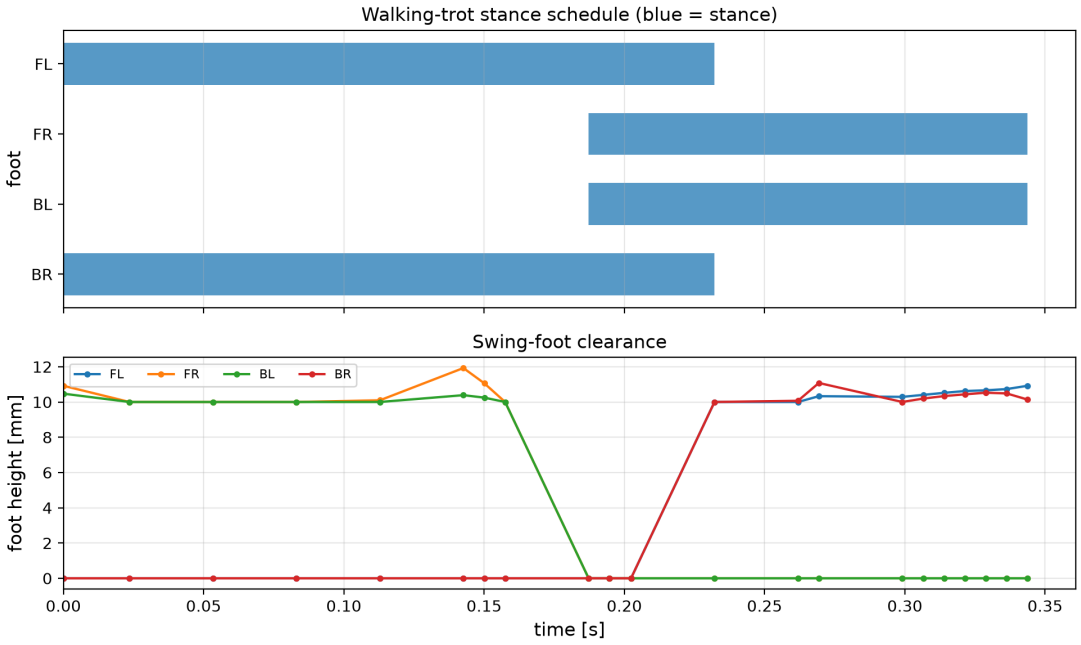


Figure 4.2: LittleDog gait: optimized contact schedule and body trajectory. The stride is 0.6875 s and the CoM height varies between 0.131 and 0.148 m.

convex sets. Both formulations use the same reachability and foot-geometry constraints; they differ in how the discrete choice is represented.

4.3.1 Exercise 5.1: Footstep Planning via MIQP

The MIQP has three groups of decision variables for a plan with N steps:

- $\mathbf{p}_L[t], \mathbf{p}_R[t] \in \mathbb{R}^2$ for $t = 0, \dots, N$: the horizontal positions of the left and right foot;
- $b_L[t,i], b_R[t,i] \in \{0,1\}$: $b_L[t,i] = 1$ if the left foot is on stone i at step t (similarly for the right foot);
- $f \in \{0,1\}$: $f = 1$ if the left foot moves first.

The constraints are

$$\begin{aligned}
\mathbf{p}_L[0] &= \mathbf{c}_{\text{init}}, & \mathbf{p}_R[0] &= \mathbf{c}_{\text{init}} - [0, 0.2]^\top, \\
\mathbf{p}_L[N] &= \mathbf{c}_{\text{goal}}, & \mathbf{p}_R[N] &= \mathbf{c}_{\text{goal}} - [0, 0.2]^\top, && (4.1)\\
\left|\mathbf{p}_L[t] - \mathbf{p}_R[t]\right| &\le \tfrac{s}{2}\mathbf{1}, & \left|\mathbf{p}_j[t+1] - \mathbf{p}_j[t]\right| &\le s\,\sigma_j(t), && (4.2)\\
\textstyle\sum_i b_L[t,i] &= 1, & \textstyle\sum_i b_R[t,i] &= 1, && (4.3)\\
A_i\mathbf{p}_L[t] &\le \mathbf{b}_i + M(1 - b_L[t,i]), & A_i\mathbf{p}_R[t] &\le \mathbf{b}_i + M(1 - b_R[t,i]), && (4.4)
\end{aligned}$$

where s is the step span, $\sigma_j(t) \in \{f, 1-f\}$ alternates with the step parity so that only one foot moves per step, and M is a big-M constant large enough to relax the stone constraint when $b_j[t,i] = 0$. Equation (4.4) is the only coupling between the continuous positions and the binaries: if the foot is assigned to stone i, its position must satisfy the stone's half-plane description $A_i\mathbf{p} \le \mathbf{b}_i$. The objective penalises the squared step lengths,

$$\min \sum_{t=0}^{N-1} \Big(\|\mathbf{p}_L[t+1] - \mathbf{p}_L[t]\|^2 + \|\mathbf{p}_R[t+1] - \mathbf{p}_R[t]\|^2 \Big),$$

and the resulting problem is solved by branch and bound with OSQP at each node. On the two benchmark terrains the MIQP finds costs 4.1593 and 5.6721 and passes the autograder 16/16.

4.3.2 Exercise 5.2: Understanding the Zero-Moment Point

This is an analytic exercise. It derives the ZMP for a free-body diagram, first without an ankle actuator and then with an ideal ankle actuator, and uses the result to discuss foot design. The ZMP is determined by the contact wrench and the CoM acceleration, not simply by the centre of pressure; the ankle torque shifts it inside the support polygon. The exercise also shows the range of motion of the actuated ankle for a given CoM trajectory (about 76° to 99° in the numerical example).

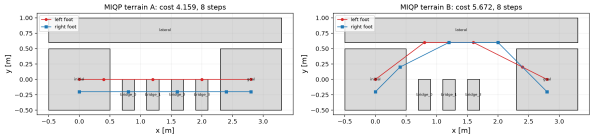
4.3.3 Exercise 5.3: Footstep Planning via GCS

The GCS formulation keeps the same geometry but replaces the binaries by graph structure. Each stepping stone i becomes a convex vertex $H_i = \{\mathbf{x} : A_i\mathbf{x} \le \mathbf{b}_i\}$, and because a foot may occupy the same stone more than once, the stone is copied $\lfloor L_i/s \rfloor + 2$ times (L_i is the stone length); copies of the same stone are joined by edges so that a foot can cross it in several steps. An edge $e = (u, v)$ exists only if

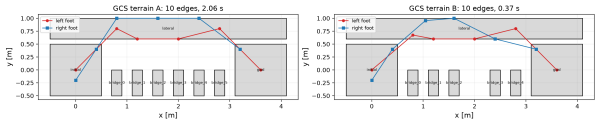
$$\|\mathbf{x}_u - \mathbf{x}_v\|_\infty \le \tfrac{s}{2}, \qquad \text{cost}(e) = 1, \tag{4.5}$$

and the problem is a shortest path from the initial vertex to the goal vertex. The solver applies a convex relaxation of the bilinear edge constraints and then rounds at most 10 candidate paths (`max_rounded_paths = 10`), returning the cheapest integer-feasible path with SNOPT. The GCS solution uses 10-edge paths and passes the autograder 16/16.

The trade-off is the one described in the notes [3]: the MIQP uses $O(N \cdot n_{\text{stones}})$ binaries and branches on them, while GCS uses a graph whose size grows with the number of stone copies but whose relaxation is often very tight. For these small terrains the MIQP is simpler to write; GCS scales better when the environment is naturally a set of convex pieces.



(a) MIQP footstep plan.



(b) GCS footstep plan.

Figure 4.3: Section 5.8 footstep plans on the benchmark terrains. The MIQP costs are 4.159 and 5.672 for terrains A and B; the GCS plans use 10 edges.

4.4 Implementation notes

I ran the ZMP and LittleDog notebooks and the two footstep-planning notebooks. The LittleDog notebook's default SNOPT iteration limit was too small for the gait to converge: the first run returned an infeasible point with the default setting. I raised the major-iteration limit to 100, after which the stride converged to 0.6875 s; the result in this report is from the modified setting. For the footstep exercises, the MIQP is solved by branch and bound with OSQP at each node, while GCS uses the default preprocessing, a convex relaxation, and `max_rounded_paths = 10` with SNOPT for the rounded paths. Both benchmark terrains pass 16/16. The ZMP exercise is analytic; the numerical values come from `zmp_exercise_analysis.py`.

4.5 Discussion

The ZMP planner, the LittleDog optimisation and the footstep planners address different levels of the same problem. The ZMP planner gives a CoM reference but assumes a fixed contact sequence; the LittleDog optimisation chooses the timing and contacts for a periodic gait; MIQP and GCS choose discrete footholds.

Each level has assumptions that are easy to forget. The ZMP planner uses the linear inverted pendulum: constant CoM height, no angular momentum about the CoM, and a flat, rigid ground. LittleDog's centroidal model ignores the individual joint torques and assumes that the contacts can supply whatever wrench the optimiser asks for. The footstep planners treat each foot as a point and use a big-M or graph abstraction for the stones; they do not check whether the leg can actually reach the chosen foothold. The reported costs (4.1593 and 5.6721 for MIQP, 10-edge paths for GCS) are therefore best read as benchmark values, not as universal performance numbers.

The parameter that matters most is the step span s: it sets both the reachability limits and the edge set of the graph, and a larger s makes the problem easier to satisfy but produces longer,

less natural steps. A next step would be a whole-body MPC that keeps the contact schedule as a decision variable and adds joint limits and torque bounds, which is the direction the later chapters of the manipulation notes take.

4.6 Answers to the notebook questions

What is the MIQP–GCS trade-off? The MIQP keeps the natural continuous variables and adds one binary per (foot, step, stone) plus one for the first step; it is easy to write and solves well for a few dozen binaries. GCS removes the binaries by duplicating each convex set and searching a graph; its size grows with the number of copies rather than the number of stones, and its convex relaxation is usually tight, so it degrades more gracefully when the terrain has many stones. For the two benchmark terrains both give feasible plans; the MIQP is the simpler model, GCS the more scalable one.

Why must the ZMP lie inside the support polygon? The contact wrench must have a non-negative normal component and must lie in the friction cone; the point of application of the resultant ground force is the centre of pressure, and it can only lie inside the convex hull of the contact points. The ankle torque shifts the ZMP within that hull but cannot move it outside without the foot tipping.

Why does LittleDog need a contact schedule? The centroidal dynamics depend on which feet are on the ground, and the contact pattern is discrete. Optimising only the joint trajectories for a fixed pattern cannot change the gait; including the timing and the contact sequence is what allows the optimiser to find a periodic trot rather than just track a prescribed one.

Why is the ZMP error after the push so small? The nominal plan is an open-loop reference; the reported 1.5×10^{-5} m is the error of the feedback simulation after the velocity push, in which the controller re-plans the CoM and the feet. It is a statement about the closed loop, not about the open-loop plan, whose tracking error is 0.153 m.

Part II

Manipulation

Chapter 5

Manipulation Ch. 1: Introduction to Manipulation and Drake

This chapter covers Examples 1.1–1.3 and Section 1.7 of *Robotic Manipulation* [4]; the corresponding reading is §1.1–1.4 (scope and simulation) and §1.5–1.7 (Drake systems and exercises), together with the course lecture [6]. The notebooks are in `Ch-1/examples/` and `Ch-1/exercises/`. Example 1.1 is included as background: the assignment lists Example 1.2 (3D teleoperation) and Example 1.3 (system diagrams), and the 2D version is the natural first step.

5.1 Examples 1.1 and 1.2: Teleoperation in 2D and 3D

The Manipulation introduction starts with teleoperation [4]. The notebook builds a HardwareStation with an IIWA14 and a WSG gripper, adds a differential inverse-kinematics block and connects a set of pose sliders. The data flow is simple: the sliders provide a desired end-effector pose, differential IK converts it into joint positions, the station's position controller tracks the joint command, and the measured pose is fed back to the sliders.

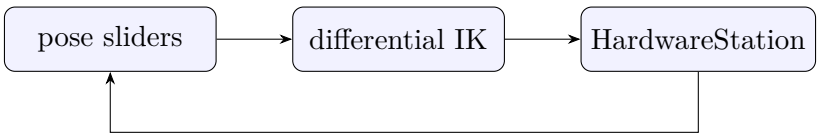


Figure 5.1: Teleoperation data flow. The sliders command a full pose, diff-IK converts it to joint positions, and the measured pose is fed back.

5.1.1 Background: Example 1.1 – Teleoperation in 2D

The 2D example restricts the visible task to roll, x and z; the planar IIWA14 has three actuated joints, so the task and the arm are matched in the nominal configuration.

5.1.2 Example 1.2: Teleoperation in 3D

The 3D example uses the full 7-DoF IIWA14 and a 6-DoF pose slider. The arm is now redundant: many joint configurations produce the same end-effector pose, and differential IK chooses the one closest to the current configuration. In both cases the sliders and the gripper button are the operator interface; the controller and the plant are hidden behind the station ports.

The differential IK solves a local least-squares problem

$$\min_{\dot{q}} \ \|J(q)\dot{q} - \dot{X}_d\|^2 + \lambda\|\dot{q}\|^2$$

subject to joint limits and integrates $\dot{q}$. It is fast and degrades gracefully: when a target is outside the workspace or near a singularity, the solver reports `stuck` instead of producing a discontinuous jump. The 3D demo also shows the limitation of purely kinematic teleoperation: the clutter is not avoided, because obstacle avoidance is a separate layer.

5.2 Example 1.3: System Diagrams

The System Diagrams example makes the modelling paradigm explicit. A manipulation system is a `Diagram` of smaller systems, each with its own state and ports. The teleoperation demo is already such a diagram: a plant, a controller, a visualizer, and a slider interface. The interactive example lets the reader expand the `iiwa_controller` block and see the hidden complexity inside a single box.

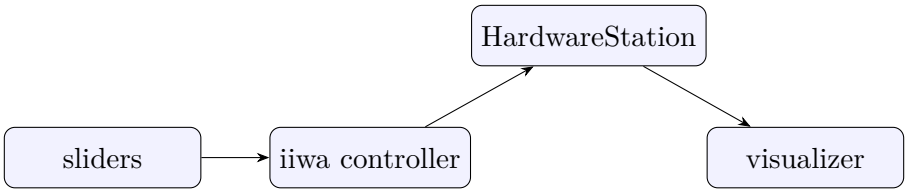


Figure 5.2: A system diagram in the spirit of Example 1.3. The station is a nested diagram; expanding the IIWA controller shows the position controller and model driver inside.

The benefit of the block-diagram approach is composability: a controller, filter or plant can be replaced without rewriting the rest of the system. The cost is that state and ports must be declared explicitly, which can feel like overhead for a small script. For a manipulation stack with perception, planning, control and simulation, the structure is what keeps the parts interchangeable. The HardwareStation scenario used in Exercise 1.4 is a declarative description of the same idea.

5.3 Section 1.7: Exercises 1.1–1.4

Exercise 1.1: Familiarize yourself with Drake. The three documentation questions have short answers:

1. In the `dynamical_systems` tutorial [5] the initial condition of `SimpleContinuousTimeSystem` is set to $x(0) = 0.9$.
2. In the C++ doxygen page on spatial vectors, angular acceleration is written `alpha`; angular velocity is `w`, torque is `t`, and the translational quantities are `v`, `a` and `f`.
3. The Python fitted-value-iteration unit test sets `options.convergence_tol = 1.0`. The default value of `DynamicProgrammingOptions` is `1e-4`, but the test deliberately uses the larger tolerance.

Exercise 1.2: Drake Systems Fundamentals. The exercise implements an inverted pendulum as a `LeafSystem` with state $[\theta, \dot{\theta}]$, a torque input and an angle output. The dynamics are $\ddot{\theta} = -g\sin\theta + \tau/(ml^2)$; for the default $m = l = 1$ this is $-9.81\sin\theta + \tau$. The system is wired into a

diagram with a constant torque source and a logger, and simulated with Drake's `Simulator`. The two verification cases give $\theta(2.5) = 0.0052$ rad and $\theta(1.8) = -0.1074$ rad. The autograder gives 70/70.

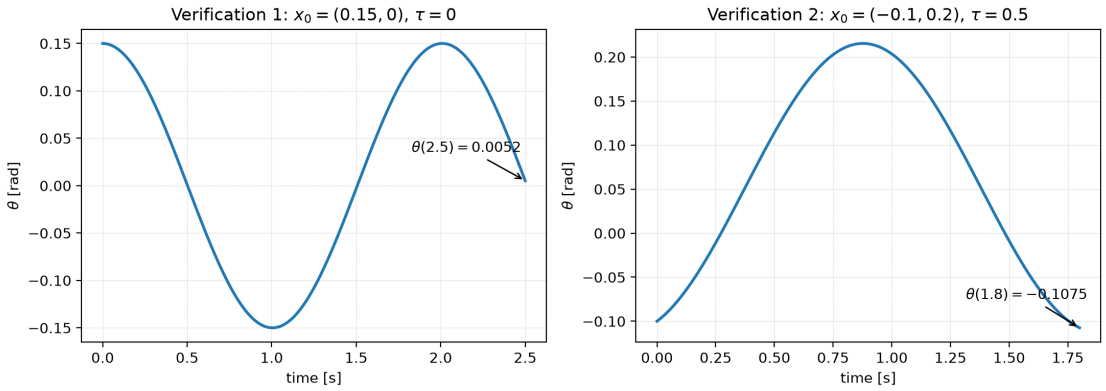


Figure 5.3: Inverted-pendulum simulations for the two verification conditions. The final angles are 0.0052 and −0.1074 rad, matching the autograder.

Exercise 1.3: Physics Simulation and Robot Loading. The exercise loads a 7-DoF IIWA14 with `Parser` and `MultibodyPlant`, welds the base, and adds a simple proportional controller as a `LeafSystem`. The controller declares one input port of size 14, which receives the plant state $[q; v]$, and one output port of size 7:

$$\tau = -K\,(q - q_d), \qquad K = 120\ \mathrm{N\,m/rad}, \quad q_d = 0.$$

Only q is used in the torque law; v is part of the port so that the same controller can later be extended with damping. The diagram connects the plant's state output to the controller and the controller's torque output back to the plant's actuation input.

The scene is then assembled from four parts: the welded IIWA14, a table authored as an SDF string, three letter assets generated by `create_sdf_asset_from_letter`, and the letters dropped onto the table. The verification run uses $q(0) = [0.2, 0.2, 0.2, 0, 0, 0, 0]$, $K = 120$, $q_d = 0$, a simulation time of 10 s and `set_target_realtime_rate=False`. The result is

$$q_{\text{final}} = [0.0126, 0.0081, -0.0573, -0.0244, -0.0001, 0.0005, 0.0000],$$

which is the value the autograder checks to 10^{-4} rad. The small residuals are due to gravity and joint damping; the P controller does not compensate gravity, so a steady-state offset is expected. The visible autograders give 10/10 for the IIWA14 structure, 10/10 for the controller, 10/10 for the full-system run, and 20/20 for the Gradescope verification, for a total of 50/50.

Exercise 1.4: HardwareStation and Advanced Scenarios. This exercise replaces the manual wiring with a HardwareStation and a YAML scenario. The scenario contains two IIWA14 robots, a local table, three letters and a free YCB object. The station is added to a diagram and simulated. The autograders give 10/10 for the basic station, 10/10 for the bimanual station, 15/15 for the full-system structure with eight model instances, and 15/15 for the simulation run, for a total of 50/50.

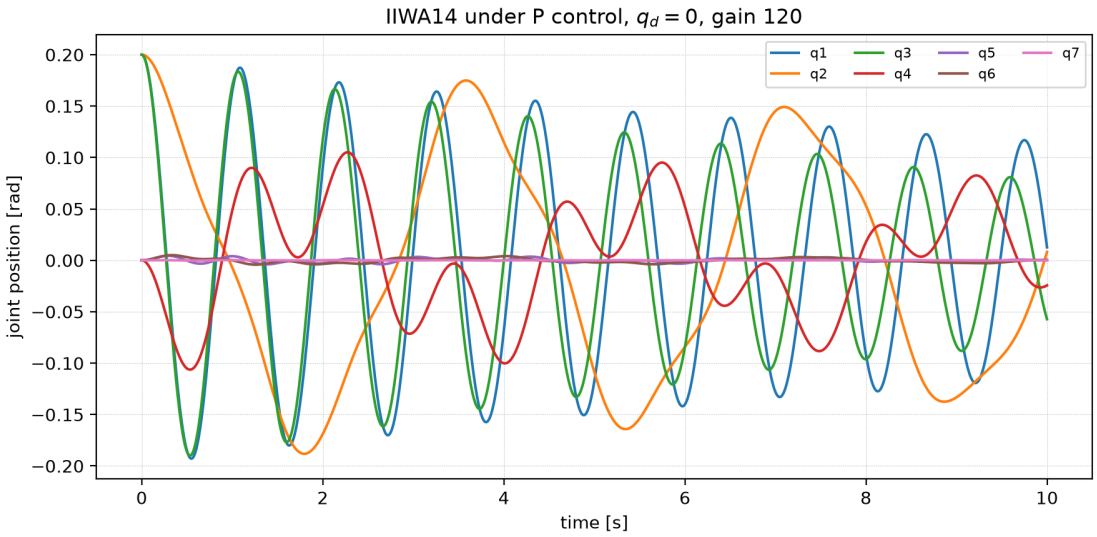


Figure 5.4: Joint positions of the IIWA14 under the proportional controller. The residuals are below 0.06 rad; the P controller does not compensate gravity, so a small steady-state offset remains.

Listing 5.1: Scenario directives for the bimanual station (abridged).

```
directives:
- add_model:
    name: iiwa_left
    file: package://drake_models/iiwa_description/urdf/iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_left::iiwa_link_0
- add_model:
    name: iiwa_right
    file: package://drake_models/iiwa_description/urdf/iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_right::iiwa_link_0
    X_PC:
        translation: [0.0, 1.0, 0.0]
- add_model:
    name: table
    file: file:///.../assets/table.sdf
- add_weld:
    parent: world
    child: table::table_link
    X_PC:
        translation: [0.5, 0.0, -0.05]
```

5.4 Implementation notes

I used the pinned `drake_models` package for the IIWA14, the WSG gripper and the YCB object. The package was not present in the initial environment, and the download from GitHub and S3

timed out on this network. I downloaded the pinned archive through a mirror, extracted it into the local Drake package-map cache, and checked it by running the official model verification; after that all notebooks resolved `package://drake_models`.

This mattered for Exercise 1.3. My first attempt used a stripped local IIWA14 URDF, and the verification failed: the final joint vector did not match to 10^{-4} rad. The simulation itself ran, so the error was not obvious from the plots. Switching to the official `drake_models` IIWA14 (with its original inertias and damping) made the verification pass 20/20, and the final vector became the one quoted in §5.3.

Two smaller issues are worth recording. The teleoperation notebook was run with `COLAB_TESTING=1`, and the 2D and 3D functions were simulated for 0.1 s. In Exercise 1.4 the scenario parser rejected a `pathlib.Path.as_uri()` string because of percent-encoding; using `"file://" + str(absolute_path)` fixed it. The exercise notebooks use the local table and letter assets, and the autograders give 70/70, 50/50 and 50/50.

5.5 Discussion

The teleoperation stack is deliberately local: differential IK tracks a pose, but it does not avoid obstacles and it can get stuck near singularities. When the target leaves the workspace the solver reports `stuck` rather than jumping, which is the right failure mode but not a recovery strategy. The System Diagrams example explains why the stack is organised as a diagram rather than a single script; the HardwareStation scenario makes the same structure declarative. The P controller in Exercise 1.3 is also intentionally simple and leaves a gravity-induced steady-state error of up to 0.06 rad.

The simulation also omits the parts of manipulation that are hardest on hardware: contact friction, object mass and pose uncertainty, perception noise, and actuator dynamics. The scenario is deterministic, so a result that passes here is a check of the wiring, not evidence of robustness. For a real manipulation system, the next steps would be a trajectory planner, a collision-aware IK or MPC layer, a gravity-compensating controller, and a closed-loop grasp controller instead of a fixed gripper command.

5.6 Answers to the notebook questions

The three documentation questions of Exercise 1.1. The initial condition in the `dynamical_systems` tutorial is $x(0) = 0.9$; the C++ spatial-vector notation for angular acceleration is `alpha`; and the fitted-value-iteration unit test uses `options.convergence_tol = 1.0` rather than the class default `1e-4`. The answers are repeated in Exercise 1.1 above with the context in which each value appears.

Why can differential IK get stuck? It solves a local least-squares problem for $\dot{q}$ and integrates it. Near a kinematic singularity the Jacobian loses rank in some directions and the commanded twist cannot be realised; at a joint limit the step is clipped. In both cases the solver makes no progress and reports `stuck` instead of producing a jump.

Why does the P controller leave a steady-state error? At equilibrium the torque is $-K(q - q_d)$, and gravity requires a non-zero torque, so q must differ from q_d to hold the arm up. The residual is $K^{-1}\tau_{\text{gravity}}$; with $K = 120$ it is small (below 0.06 rad) but not zero. Adding an integral term or a gravity feedforward would remove it.

Why use HardwareStation instead of manual wiring? The manual version in Exercise 1.3 shows exactly what is connected to what, which is useful once. The scenario version in Exercise 1.4 puts the same information in a YAML file, so the plant, the robots and the objects can be changed without touching code. The cost is that the connections are less visible; the benefit is that the same diagram can be rebuilt from data.

Chapter 6

Conclusions

The four chapters share a common methodological pattern. A model is written in terms of state and inputs; a local linearisation is used to design a controller; the nonlinear simulation is used to find the limits of the linear design. The limits appear in different forms in each chapter: saturation and a finite basin for the acrobot and cart-pole; impact losses and a narrow basin for the compass gait; energy conservation and partial stability for SLIP; and contact planning for the legged robots.

The most consistent lesson is that underactuation and controllability answer different questions. The acrobot, cart-pole and quadrotor are all underactuated, yet their linearisations are controllable. What changes from one system to the next is how much input authority is available in each direction, and the singular values and nonlinear simulations show that this difference can be the difference between a comfortable recovery and a saturated failure.

A second observation is that LQR is a local tool. The closed-loop eigenvalues give local exponential stability, but they say little about the basin of attraction or about what happens when the actuator saturates. The acrobot and cart-pole runs make this concrete: the same controller that recovers from a small perturbation fails from a larger one or from a different direction.

The hybrid systems in Chapter 4 require a different object. The limit cycle and its return map replace the fixed point and the Jacobian. Impact maps are dissipative and can produce a stable gait, but the basin can be narrow; a conservative model such as SLIP gives a one-dimensional apex map and only partial stability. The hopper controller is a good example of using the energy balance directly instead of trying to track a trajectory.

Finally, planning and control are complementary. The ZMP planner, the LittleDog optimisation, MIQP and GCS all choose a contact sequence or a reference trajectory; the feedback controller only tracks it. The software stack—Drake diagrams, URDF/SDF assets, differential IK and HardwareStation—is what makes it possible to move between these levels without rewriting the whole system.

Bibliography

[1] R. Tedrake, *Underactuated Robotics*, Chapter 3: Acrobots, Cart-Poles, and Quadrotors. Assignments: Examples 3.1, 3.2, 3.5, 3.6 and §3.8. https://underactuated.csail.mit.edu/acrobot.html#section8

[2] R. Tedrake, *Underactuated Robotics*, Chapter 4: Simple Models of Walking and Running. Assignments: Examples 4.3–4.6 and §4.6. https://underactuated.csail.mit.edu/simple_legs.html#section6

[3] R. Tedrake, *Underactuated Robotics*, Chapter 5: Highly-articulated Legged Robots. Assignments: Examples 5.1–5.2 and §5.8. https://underactuated.csail.mit.edu/humanoids.html#section8

[4] R. Tedrake, *Robotic Manipulation*, Chapter 1: Introduction. Assignments: Examples 1.2–1.3 and §1.7. https://manipulation.csail.mit.edu/intro.html#section7

[5] Drake tutorials, including the dynamical systems and authoring leaf systems tutorials, and the C++ doxygen page on spatial vectors. https://github.com/RobotLocomotion/drake/tree/master/tutorials, https://drake.mit.edu/doxygen_cxx/group__multibody__spatial__vectors.html

[6] Course lectures: 6.8210 Spring 2024 Lecture 5, "Acrobots, Cart-poles, and Quadrotors I" https://www.youtube.com/watch?v=UBPL0IbyJy4; 6.832 Spring 2022 Lecture 15, "Simple Models of Walking" https://www.youtube.com/watch?v=N37FMfOioK0; 6.8210 Spring 2024 Lecture 16, "Humanoid Robots" https://www.youtube.com/watch?v=cRu4EqBswbk; 6.4210 Fall 2023 Lecture 1, "Introduction: Anatomy of a Manipulation System" (course YouTube playlist).

Appendix A

Code and Reproduction

The code is organised by chapter:

Chapter	Main files
Ch. 3	`Ch-3/Ch3_Assignment_Report/code/`
Ch. 4	`Ch-4/Ch4_Assignment_Report/code/`
Ch. 5	`Ch-5/Ch5_Assignment_Report/code/`
Ch. 1	`Ch-1/Ch1_Assignment_Report/code/`

The notebooks can be executed headlessly with

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=300 \
  <notebook>.ipynb
```

and the reports are compiled with `pdflatex` run twice. The combined code is in `Assignment_1_Code.zip`.

A.1 Selected code

Cart-pole dynamics and LQR.

```
def f(x, u):
    c, s = np.cos(x[1]), np.sin(x[1])
    g = 9.81
    return np.array([
        x[2], x[3],
        (u[0] + s * (x[3]**2 + g * c)) / (1 + s**2),
        (-u[0] * c - x[3]**2 * c * s - 2 * g * s) / (1 + s**2)])
```

Double-pendulum URDF (the part that differs from the single pendulum).

```
<joint name="theta0" type="continuous">
  <parent link="base" />
  <child link="pendulum0" />
  <origin xyz="0 0 0" />        <!-- pivot at the cart centre -->
  <axis xyz="0 -1 0" />
</joint>

<link name="pendulum0">
```

```
  <inertial>
    <origin xyz="0 0 -1" />     <!-- ball 1 m below the pivot -->
    <mass value="1" />
  </inertial>
</link>

<joint name="theta1" type="continuous">
  <parent link="pendulum0" />
  <child link="pendulum1" />
  <origin xyz="0 0 -1" />       <!-- at the ball of pendulum0 -->
  <axis xyz="0 -1 0" />
</joint>

<link name="pendulum1">
  <inertial>
    <origin xyz="0 0 1" />      <!-- ball 1 m from the second joint -->
    <mass value="1" />
  </inertial>
</link>
```

Note the sign change between the two links: the first ball is at $z = -1$ in `pendulum0`, the second at $z = +1$ in `pendulum1`. That is what makes $\theta_0 = \theta_1 = \pi$ the upright configuration. The inertias are zero in this exercise because each link is a point mass; the ball and rod are visual elements only.

Hopper controller (Chapter 4, Section 4.6).

```
class PreloadController(LeafSystem):
    def __init__(self, h):
        LeafSystem.__init__(self)
        self.DeclareVectorInputPort("state", 4)
        self.DeclareVectorOutputPort("delta", 1, self.DoCalcVectorOutput)
        self.h = h
        self.hopper_parameters = get_hopper_parameters()
        self.last_delta = 0.0
        self.last_state = np.full(4, np.nan)

    def is_bottom_or_apex(self, state):
        if np.any(np.isnan(self.last_state)):
            return None
        zb, l, zb_dot, l_dot = state
        last_zb_dot = self.last_state[2]
        if last_zb_dot <= 0.0 and zb_dot > 0.0:
            return "bottom" if zb < 0.0 else "apex"
        if last_zb_dot >= 0.0 and zb_dot < 0.0:
            return "apex" if zb > 0.0 else "bottom"
        return None

    def get_delta_bar(self, state):
        k, mb, ml, g = self.hopper_parameters
        zb = state[0]
        return (0.5 * zb + mb * g / k
                - ((mb + ml) ** 2) * g * self.h / (k * mb * zb))

    def DoCalcVectorOutput(self, context, delta):
        state = self.get_input_port(0).Eval(context)
```

```
        event = self.is_bottom_or_apex(state)
        if event == "bottom":
            delta.SetFromVector([max(self.get_delta_bar(state), 0.0)])
        elif event == "apex":
            delta.SetFromVector([0.0])
        else:
            delta.SetFromVector([self.last_delta])
        self.last_delta = delta[0]
        self.last_state = np.array(state)
```

Cart-pole LQR wiring (Chapter 3).

```
def cartpole_balancing(cartpole_urdf, x_star, Q, R):
    builder = DiagramBuilder()
    plant, scene_graph = AddMultibodyPlantSceneGraph(builder, time_step=0.0)
    Parser(plant).AddModelsFromString(cartpole_urdf, "urdf")
    plant.Finalize()
    plant.set_name("cartpole")

    context = plant.CreateDefaultContext()
    context.get_mutable_continuous_state_vector().SetFromVector(x_star)
    plant.get_actuation_input_port().FixValue(context, [0])
    input_i = plant.get_actuation_input_port().get_index()

    lqr = LinearQuadraticRegulator(
        plant, context, Q, R, input_port_index=input_i)
    lqr = builder.AddSystem(lqr)
    builder.Connect(plant.get_state_output_port(), lqr.get_input_port(0))
    builder.Connect(lqr.get_output_port(0),
                    plant.get_actuation_input_port())
    return builder.Build()
```

Inverted pendulum system (Chapter 1).

```
class InvertedPendulum(LeafSystem):
    def __init__(self, mass=1.0, length=1.0, gravity=9.81):
        LeafSystem.__init__(self)
        self.mass, self.length, self.gravity = mass, length, gravity
        self.DeclareContinuousState(2)
        self.torque_port = self.DeclareVectorInputPort("torque", 1)
        self.DeclareVectorOutputPort("theta", 1, self.OutputTheta)

    def DoCalcTimeDerivatives(self, context, derivatives):
        x = context.get_continuous_state_vector()
        theta, theta_dot = x[0], x[1]
        torque = self.torque_port.Eval(context)[0]
        theta_ddot = -self.gravity * np.sin(theta) + torque / (
            self.mass * self.length**2)
        derivatives.get_mutable_vector().SetFromVector(
            [theta_dot, theta_ddot])

    def OutputTheta(self, context, output):
        output.SetFromVector([context.get_continuous_state_vector()[0]])
```

Hopper controller logic (Chapter 4).

```
def is_bottom_or_apex(self, state):
    if np.any(np.isnan(self.last_state)):
        return None
    zb, l, zb_dot, l_dot = state
    last_zb_dot = self.last_state[2]
    if last_zb_dot <= 0.0 and zb_dot > 0.0:
        return "bottom" if zb < 0.0 else "apex"
    if last_zb_dot >= 0.0 and zb_dot < 0.0:
        return "apex" if zb > 0.0 else "bottom"
    return None

def DoCalcVectorOutput(self, context, delta):
    state = self.get_input_port(0).Eval(context)
    event = self.is_bottom_or_apex(state)
    if event == "bottom":
        delta.SetFromVector([max(self.get_delta_bar(state), 0.0)])
    elif event == "apex":
        delta.SetFromVector([0.0])
    else:
        delta.SetFromVector([self.last_delta])
    self.last_delta = delta[0]
    self.last_state = np.array(state)
```

HardwareStation scenario. The scenario in Exercise 1.4 uses `add_model`, `add_weld` and `default_free_body_pose` directives; the local table and letters are referenced by absolute `file://` URIs, while the robots and the YCB object use `package://drake_models`.

MIQP footstep planner (Chapter 5, Exercise 5.1).

```
def add_decision_variables(prog, terrain, n_steps):
    n_stones = len(terrain.stepping_stones)
    position_left = prog.NewContinuousVariables(rows=n_steps + 1, cols=2)
    position_right = prog.NewContinuousVariables(rows=n_steps + 1, cols=2)
    stone_left = prog.NewBinaryVariables(rows=n_steps + 1, cols=n_stones)
    stone_right = prog.NewBinaryVariables(rows=n_steps + 1, cols=n_stones)
    first_left = prog.NewBinaryVariables(1)[0]
    return position_left, position_right, stone_left, stone_right, first_left

def footstep_planner(terrain, n_steps, step_span):
    prog = MathematicalProgram()
    decision_variables = add_decision_variables(prog, terrain, n_steps)

    set_initial_and_goal_position(prog, terrain, decision_variables)
    relative_position_limits(prog, n_steps, step_span, decision_variables)
    step_sequence(prog, n_steps, step_span, decision_variables)
    one_stone_per_foot(prog, n_steps, decision_variables)
    foot_in_stepping_stone(prog, terrain, n_steps, decision_variables)
    minimize_step_length(prog, n_steps, decision_variables)

    bb = MixedIntegerBranchAndBound(prog, OsqpSolver().solver_id())
    result = bb.Solve()
    if result != result.kSolutionFound:
        raise ValueError("Infeasible optimization problem.")
```

```
    decision_variables_opt = [bb.GetSolution(v) for v in decision_variables]
    return decision_variables_opt, bb.GetOptimalCost()

def minimize_step_length(prog, n_steps, decision_variables):
    position_left, position_right = decision_variables[:2]
    for t in range(n_steps):
        d_left = position_left[t + 1] - position_left[t]
        d_right = position_right[t + 1] - position_right[t]
        prog.AddQuadraticCost(d_left.dot(d_left))
        prog.AddQuadraticCost(d_right.dot(d_right))
```

GCS footstep planner (Chapter 5, Exercise 5.3).

```
def add_edge_constraints_costs(G, E_list, step_span):
    E = []
    for u, v in E_list:
        e = G.AddEdge(u, v)
        for k in range(2):   # |x_u - x_v|_inf <= step_span / 2
            e.AddConstraint(e.xu()[k] - e.xv()[k] <= step_span / 2.0)
            e.AddConstraint(e.xv()[k] - e.xu()[k] <= step_span / 2.0)
        e.AddCost(1.0)       # every edge is one step
        E.append(e)
    return G, E

def GCS_footstep_planner(T, step_span, convex_relaxation, solver):
    G = GraphOfConvexSets()
    G, V, E_list = add_vertex_and_edges_for_set_copies(G, T, step_span)
    G, E_list = add_edges_towards_parent_vertices(G, V, E_list)
    G, E = add_edge_constraints_costs(G, E_list, step_span)

    options = GraphOfConvexSetsOptions()
    options.preprocessing = True
    options.max_rounded_paths = 10
    options.convex_relaxation = convex_relaxation
    options.solver = solver
    result = G.SolveShortestPath(V[0], V[1], options)
    path, path_position = extract_path(V, E, result)
    return path_position, len(path) - 1
```